

บทที่ 3

การออกแบบระบบ

ในหัวข้อนี้กล่าวถึงการออกแบบชิ้นงานวิจัย หรือการออกแบบส่วนต่างๆ ของโครงการนี้

3.1 บทนำ

โครงการชิ้นนี้นำหลักการของออกแบบเชิงวัตถุ หลักการออกแบบเกม ความรู้ด้านปัญญาประดิษฐ์ และความรู้ด้านการติดต่อสื่อสารมาประกอบการออกแบบเกม

3.2 การออกแบบรายละเอียดของเกม

3.2.1 เนื้อเรื่องภายในเกม

ในอดีตดวงดาวดวงหนึ่งมีสัตว์เทพเจ้าอยู่ 4 ชนิด ได้แก่ พืช, มังกร, นกไฟ และเต่ายักษ์ โดยสัตว์เทพทั้ง 4 นี้เป็นผู้ดูแลดวงดาวดวงนั้นมาช้านาน เมื่อเวลาผ่านไปมาจนถึงจุดเปลี่ยนแปลงครั้งใหญ่ของดวงดาวแห่งนี้ในวันที่สรรพสิ่งต่างๆ บนดวงดาวจะถูกธรรมชาติทำลายจนหมดสิ้น เพื่อเป็นการก่อกำเนิดสิ่งต่างๆ ขึ้นใหม่ จากเหตุการณ์ครั้งนั้นทำให้สัตว์เทพเจ้าทั้ง 4 ชนิดได้หายไปเหลือไว้เพียงแต่ “สายเลือด” ที่ตกทอดไปยังสิ่งมีชีวิตที่เกิดขึ้นใหม่ ดังนั้นสิ่งมีชีวิต หรือมอนสเตอร์ที่เกิดขึ้นใหม่ภายในดวงดาวแห่งนี้ประกอบไปด้วยสายเลือดของสัตว์เทพเจ้าทั้ง 4 ผสมผสานอยู่ในกายมายน้อยแตกต่างกันไป

ผู้เล่นรับบทเป็นมนุษย์โลกที่เดินทางไปยังดวงดาวแห่งนี้และเก็บมอนสเตอร์จากดาวดวงนั้นกลับมาเลี้ยงที่โลกมนุษย์ เป็นสัตว์เลี้ยง และเป็นเพื่อนกับผู้เล่นตลอดเวลา

ผู้เล่นสามารถเลี้ยงดูมอนสเตอร์ของตนแล้วนำไปผสมพันธุ์กับมอนสเตอร์ของผู้เล่นคนอื่นเพื่อให้ได้มอนสเตอร์ชนิดใหม่ที่มีคุณสมบัติที่ดีเยี่ยมยิ่งขึ้นกลับมาเลี้ยง หรือสามารถนำมอนสเตอร์ของตนไปต่อสู้ประลองฝีมือกับมอนสเตอร์ของผู้เล่นคนอื่นเพื่อวัดความสามารถในการเลี้ยงดูก็ได้

การต่อสู้ภายในเกมผู้เล่นสามารถต่อสู้ตัวคนเดียว หรือต่อสู้กับผู้เล่นด้วยกันได้ด้วยการเชื่อมต่อแบบ wi-fi

เส้นทางสู่การเป็นนักเลี้ยงมอนสเตอร์มือหนึ่ง กำลังเปิดรอผู้เล่นอยู่...

3.2.2 วัตถุประสงค์ของเกม

วัตถุประสงค์ของเกม คือ การเลี้ยงดูมอนสเตอร์ของตนเองให้มีความแข็งแกร่งเหนือมอนสเตอร์อีกมากมายหลากหลายชนิดที่มีอยู่ในเกม และมีความแข็งแกร่งเหนือกว่ามอนสเตอร์ของผู้เล่นอื่นๆ ซึ่งผู้เล่นต้องแสวงหาพ่อพันธุ์-แม่พันธุ์ที่แข็งแกร่งจึงจะก่อกำเนิดมอนสเตอร์ที่แข็งแกร่งที่สุดได้

3.2.3 มุมมองภายในเกม

มุมมองภายในเกมเป็นมุมมองด้านหน้า บนหน้าจออุปกรณ์ฟ็อกเก็ตพีซี ขนาด 640 x 480 พิกเซล ด้วยการแสดงผลแบบ 3 มิติ

3.2.4 กฎ กติกาภายในเกม

1. รายละเอียดของมอนสเตอร์

มอนสเตอร์ 1 ตัวมีรายละเอียด ดังนี้

พารามิเตอร์	รายละเอียด	หมายเหตุ
Hp	ค่าพลังชีวิต	มีค่าไม่เกิน HPmax
Hpmax	ค่าพลังชีวิตสูงสุด	-
Sp	ค่าพลังวิญญาน	มีค่าไม่เกิน Hpmax
Spmax	ค่าพลังวิญญานสูงสุด	ค่าพลังวิญญานสูงสุด
Age	อายุ	-
Exp	ค่าประสบการณ์	มีผลต่อการเลื่อนระดับ Level
Level	ระดับความเก่ง	มีค่าได้ตั้งแต่ 1 ถึง 99
Energy	ค่าพลังงาน	มีผลต่อการทำการฝึกฝน มีค่าได้ตั้งแต่ 0 ถึง 99
Status	ค่าสถานะ	มีผลต่อการเจริญเติบโต มีสถานะ {ตาย, ใกล้ตาย, ป่วย, ปกติ, มีความสุข, แข็งแรงดีมาก}
Name	ชื่อ	-
Full	ระดับความอิ่ม	มีผลต่อค่าสถานะ

		มีค่าได้ตั้งแต่ 0 ถึง 100
Thirsty	ระดับความกระหาย	มีผลต่อค่าสถานะ มีค่าได้ตั้งแต่ 0 ถึง 100
Str	ค่าความแข็งแรงของกาย	มีผลต่อค่าพลังโจมตีระยะใกล้ มีค่าได้ตั้งแต่ 1 ถึง 255
Qi	ค่าความแข็งแรงของจิต	มีผลต่อค่าพลังโจมตีระยะไกล มีค่าได้ตั้งแต่ 1 ถึง 255
Def	ค่าความทนทานของร่างกาย	ค่าพลังป้องกันการถูกโจมตี มีค่าได้ตั้งแต่ 1 ถึง 255
Agi	ค่าความคล่องตัว	มีผลต่อการหลบหลีกเมื่อถูกโจมตี มีค่าได้ตั้งแต่ 1 ถึง 255
Dex	ค่าความแม่นยำ	ส่งผลให้สามารถโจมตีได้แรงมากขึ้น มีค่าได้ตั้งแต่ 1 ถึง 255
Fit	ค่าความพร้อมของร่างกาย	ส่งผลต่อการโจมตีด้วยค่าพลังโจมตีสูงสุด มีค่าได้ตั้งแต่ 1 ถึง 255
Vit	ระดับพลังชีวิต	ส่งผลต่อค่า Hpmax มีค่าได้ตั้งแต่ 1 ถึง 255
Breath	ระดับพลังปราณ	ส่งผลต่อค่า Spmax มีค่าได้ตั้งแต่ 1 ถึง 255
Int	ค่าความฉลาด	ส่งผลต่อการเลือกการกระทำ (action) ที่ดีที่สุดในขณะต่อสู้ มีค่าได้ตั้งแต่ 1 ถึง 255
Hierarchy_level	ระดับชั้นสายพันธุ์	-
Body	รหัสร่างกาย	บอกว่ามีร่างกายแบบไหน
Type	ชนิดหลักของสายเลือด	มี 4 ชนิด ได้แก่ พยัคฆ์, มังกร, นกไฟ, เต่ายักษ์
Team	ประเภทของมอน	บอกว่าเป็นมอนสเตอร์ของผู้เล่น, ของผู้เล่นคนอื่น หรือของ

		computer
expRequired	จำนวนค่าประสบการณ์ที่ต้องใช้เพื่อเลื่อนระดับความเก่ง (Level) ในระดับถัดไป	เมื่อมีค่าประสบการณ์ (Exp) ถึงค่านี้นี้ ตัวละครจะมีระดับ Level เพิ่มขึ้นอีก 1 ระดับ
Space	ขอบเขตการเรียนรู้จากการฝึกฝน (จะเพิ่มขึ้นเมื่อระดับ Level เพิ่มขึ้น)	ค่าพารามิเตอร์ที่ใช้ในการต่อสู้ (Str, Qi, Def, Agi, Dex, Fit, Vit, Breath, Int) มีค่ารวมกันได้ไม่เกินค่า Space

ตารางที่ 3.1 แสดงค่าพารามิเตอร์ประจำตัวมอนสเตอร์

การกระทำ	รายละเอียด	หมายเหตุ
Move_Forward	เคลื่อนที่ไปข้างหน้า	-
Move_Backward	เคลื่อนที่ไปข้างหลัง	-
Jump	เคลื่อนที่หลบการโจมตี	นำค่า Agi มาคำนวณโอกาสที่จะหลบได้
Punch	ต่อย / เตะ	การโจมตีใกล้ ค่าความรุนแรงขึ้นกับค่า Str
Power	ปล่อยพลัง	การโจมตีไกล ค่าความรุนแรงขึ้นกับค่า Qi
Charge	ชาร์จพลัง	เพิ่มค่า Sp ครั้งละ 5 ค่า

ตารางที่ 3.2 แสดงการกระทำประจำตัวของมอนสเตอร์

2. ชนิดของมอนสเตอร์

ตัวละครมอนสเตอร์ภายในเกมสืบสายเลือดมาจากสัตว์เทพในอดีต 4 ชนิด ที่มีลักษณะเด่นแตกต่างกัน ดังนี้

- 1) พยัคฆ์ เน้นการโจมตีระยะใกล้รุนแรง (เน้น Str)
- 2) มังกร เน้นการโจมตีระยะไกลรุนแรง (เน้น Qi)
- 3) นกไฟ เน้นความรวดเร็วในการหลบหลีก (เน้น Agi)
- 4) เต่ายักษ์ เน้นความแข็งแกร่งป้องกันการโจมตี (เน้น Def)

มอนสเตอร์ภายในเกมมีสายเลือดทั้ง 4 ชนิดนี้ปะปนอยู่ในร่างกาย สำหรับตัวละครมอนสเตอร์ของผู้เล่นเป็นมอนสเตอร์ชนิดไหนนั้น ขึ้นอยู่กับค่าพารามิเตอร์หลักของมอนสเตอร์นั้นๆ กล่าวคือหากมอนสเตอร์มีค่าพารามิเตอร์ค่าใดใน 4 ค่าที่กล่าวถึงด้านบนมากกว่า จะถือว่ามอนสเตอร์ตัวนั้นมีสายเลือดของสัตว์เทพเจ้าที่เน้นค่าพารามิเตอร์นั้นมากที่สุด

3. การเจริญเติบโต

ตัวละครมอนสเตอร์ภายในเกมสามารถมีการเจริญเติบโต หรือมีวิวัฒนาการได้ 2 ทาง ได้แก่

- 1) การฝึกฝนเลี้ยงดูจากผู้เล่น
- 2) การผสมพันธุ์ระหว่างมอนสเตอร์จากผู้เล่นตั้งแต่ 2 คนขึ้นไป

ในกรณีที่เจริญเติบโตจากการเลี้ยงดูฝึกฝนจากผู้เล่นนั้น ระบบจะทำการตรวจสอบสถานะของมอนสเตอร์อยู่ตลอดเวลาว่ามอนสเตอร์มีการเจริญเติบโตเพียงพอหรือยัง หากการเจริญเติบโตถึงระดับหนึ่งแล้วมอนสเตอร์จะได้รับการเปลี่ยนแปลงร่างกายไปเป็นร่างกายตามชนิดสายเลือดหลักของมอนสเตอร์ในขณะนั้น

การตรวจสอบการเปลี่ยนร่างของมอนสเตอร์ทำการเปลี่ยนแปลงทุกครั้งที่ค่า Level ของมอนสเตอร์มีค่าที่ลงท้ายด้วยเลข 5 หรือเลข 0 เช่น ที่ค่า Level 5, 10, 15, 20, 25 เป็นต้น โดยการเปลี่ยนร่างในครั้งถัดไปสามารถเปลี่ยนเป็นร่างมอนสเตอร์ที่อยู่คนละสายกับร่างในปัจจุบันได้ (ตามชนิดของมอนสเตอร์ในระดับ Level นั้นจริงๆ)

4. เมนูภายในเกม

4.1 ดูสถานะของมอนสเตอร์

แสดงค่าพารามิเตอร์และสถานะของมอนสเตอร์ในสภาวะปัจจุบัน

4.2 ฝึกฝน

เมนูการฝึกฝนมอนสเตอร์ ซึ่งประกอบไปด้วยชนิดของการฝึกฝน ดังนี้

เมนูการฝึกฝน	สิ่งที่ได้รับ	หมายเหตุ
ยกดรัมเบล	เพิ่มค่า Str และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Str
งอข้อ	เพิ่มค่า Qi และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Qi
วิ่ง	เพิ่มค่า Agi และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ

		การเพิ่มค่า Agi
ลากท่อนซุง	เพิ่มค่า Def และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Def
ปาเป้า	เพิ่มค่า Dex และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Dex
กินอาหารชีวจิต	เพิ่มค่า Vit, Breath, Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Vit, Breath
กินวิตามิน	เพิ่มค่า Fit และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Fit
อ่านหนังสือ	เพิ่มค่า Int และค่า Exp	สุ่มค่าตั้งแต่ 1 ถึง 3 สำหรับ การเพิ่มค่า Int
ฝึกฝนต่อสู้	ได้รับค่า Exp	ได้รับค่า Exp สูงกว่าการ ฝึกฝนอื่นๆ จะได้ในกรณีที่ เป็นผู้ชนะ

ตารางที่ 3.3 แสดงการฝึกฝนภายในเมนูฝึกฝน

4.3 ให้อาหาร

เมนูอาหาร มีดังนี้

- | | | |
|--------------|---|-----------------------|
| 1) ไก่ย่าง | - | เพิ่มระดับความอิม +5 |
| 2) ข้าวผัดปู | - | เพิ่มระดับความอิม +10 |
| 3) สเต็ก | - | เพิ่มระดับความอิม +15 |

เมนูเครื่องดื่ม มีดังนี้

- | | | |
|-------------|---|-----------------------------|
| 1) น้ำเปล่า | - | เพิ่มระดับความไม่กระหาย +5 |
| 2) ชาเขียว | - | เพิ่มระดับความไม่กระหาย +10 |
| 3) ไวน์ | - | เพิ่มระดับความไม่กระหาย +15 |

4.4 พักผ่อน

เมนูนี้ทำการให้มอนสเตอร์พักผ่อน โดยตลอดระยะเวลาที่มอนสเตอร์อยู่ในโหมดพักผ่อนนี้ ค่าพารามิเตอร์เหล่านี้จะค่อยๆ เพิ่มขึ้นทีละน้อย

- ค่าพลังชีวิต (Hp)
- ค่าพลังวิญญาณ (Sp)

- ค่าพลังงาน (Energy)

4.5 ต่อสู้ประลองฝีมือกับผู้เล่นคนอื่น

เมนูนี้ทำการเข้าสู่การต่อสู้ระหว่างผู้เล่นกับผู้เล่นคนอื่น ซึ่งรองรับการต่อสู้ระหว่างผู้เล่นได้ครั้งละ 2 คนต่อ 1 การต่อสู้เท่านั้น โดยจะต้องระบุหมายเลข IP Address ของผู้เล่นฝ่ายตรงข้าม พร้อมทั้งเลือกจะทำให้ใครเป็นเครื่องที่ทำการต่อสู้ จากนั้นเครื่องที่ไม่ได้เป็นเครื่องสำหรับทำการต่อสู้จะส่งข้อมูลมอนสเตอร์ของตัวเองไปให้แก่เครื่องที่ทำหน้าที่จัดการต่อสู้

จากนั้นระบบจะตรวจสอบข้อมูลของมอนสเตอร์ของผู้เล่นทั้งสองคนแล้วทำการต่อสู้ เมื่อรู้ผลแพ้ชนะแล้วระบบจะเพิ่มค่าประสบการณ์ให้แก่มอนสเตอร์ที่เป็นผู้ชนะ แล้วส่งข้อมูลมอนสเตอร์กลับคืนแก่ผู้เล่นทั้งสอง และปิดการเชื่อมต่อ

4.6 การผสมพันธุ์มอนสเตอร์

เมนูนี้ทำการผสมพันธุ์มอนสเตอร์โดยรับข้อมูลพ่อพันธุ์ – แม่พันธุ์จากผู้เล่นเข้ามาในระบบ แล้วทำการผสมพันธุ์จนได้ลูกมอนสเตอร์ที่มีวิวัฒนาการสูงขึ้น แล้วส่งข้อมูลลูกมอนสเตอร์ที่ได้ในรูปแบบของไข่ (Egg) กลับคืนไปให้แก่ผู้เล่น ซึ่งมีข้อกำหนดว่าไข่ (Egg) ที่ได้สามารถส่งค่ากลับคืนไปให้แก่ผู้เล่นได้แค่ 1 คนเท่านั้น

ระบบที่จัดการการผสมพันธุ์นี้อยู่บนเครื่องเซิร์ฟเวอร์ ซึ่งผู้เล่นทุกคนที่ต้องการผสมพันธุ์มอนสเตอร์ในแต่ละครั้งต้องทำการเชื่อมต่อเข้าสู่เครื่องเซิร์ฟเวอร์นี้ โดยมีข้อจำกัดว่าระบบรองรับการเชื่อมต่อจากผู้เล่นได้ครั้งละ 5 คน / เครื่อง เท่านั้นต่อการผสมพันธุ์มอนสเตอร์แต่ละครั้ง

4.7 ห้องฟักไข่

ส่วนของการเก็บข้อมูลไข่มอนสเตอร์ หากมีข้อมูลไข่มอนสเตอร์จัดเก็บอยู่ผู้เล่นสามารถเลือกคำสั่ง ‘ฟักไข่’ ได้ภายในเมนูนี้

หากผู้เล่นทำการ ‘ฟักไข่’ มอนสเตอร์ตัวใหม่ที่ออกมาจากไข่ (Egg) จะทำการทับข้อมูลมอนสเตอร์ตัวที่ผู้เล่นมีอยู่ก่อนหน้าที่จะทำการฟักไข่ และไม่สามารถนำข้อมูลมอนสเตอร์ตัวที่ถูกลบไปแล้วกลับคืนมาได้

5. การต่อสู้ของมอนสเตอร์

การต่อสู้ภายในเกมตัวมอนสเตอร์จะทำการคิด และดำเนินการกระทำต่างๆ เอง โดยผู้เล่นไม่สามารถบังคับมอนสเตอร์ได้

ผลการต่อสู้จะสิ้นสุดลงเมื่อมอนสเตอร์ฝ่ายใดฝ่ายหนึ่งมีค่าพลังชีวิต (Hp) เท่ากับ 0 ก่อนหรือเมื่อเวลาในจากต่อสู้ลดลงจนหมด ในกรณีที่เวลาหมด มอนสเตอร์ตัวที่มีพลังชีวิต (Hp) เหลือมากกว่าจะถือว่าเป็นผู้ชนะในการต่อสู้นั้น

หลักการคิดของมอนสเตอร์ในการต่อสู้อธิบายในหัวข้อที่ ‘3.4 การออกแบบอัลกอริทึมที่ใช้ในการต่อสู้’

6. การผสมพันธุ์ของมอนสเตอร์

การผสมพันธุ์ของมอนสเตอร์มีเพื่อสร้างมอนสเตอร์รุ่นใหม่ที่มีวิวัฒนาการ และความก้าวหน้ามากกว่ารุ่นก่อน โดยการออกแบบกล่าวถึงในหัวข้อที่ ‘3.4 การออกแบบการผสมพันธุ์มอนสเตอร์’

7. การสร้างตัวละครมอนสเตอร์ศัตรู

เมื่อผู้เล่นเลือกเมนู ‘ฝึกฝนต่อสู้’ ภายในเมนู ‘ฝึกฝน’ มอนสเตอร์ของผู้เล่นจะเข้าสู่การต่อสู้กับมอนสเตอร์ศัตรูที่ระบบเป็นผู้สร้างขึ้น

ระบบทำการสร้างมอนสเตอร์ศัตรูที่มีระดับความเก่ง (Level) ระดับเดียวกับมอนสเตอร์ของผู้เล่น และทำการสุ่มชนิดและค่าพารามิเตอร์ต่างๆ ของมอนสเตอร์ศัตรู ภายในขอบเขตของค่า Space ในระดับ Level นั้นๆ

3.3 การออกแบบรูปแบบของไฟล์ที่ใช้เก็บข้อมูลตัวละคร

ข้อมูลของมอนสเตอร์ที่เป็นตัวละครของผู้เล่น มีการจัดเก็บข้อมูลในรูปแบบของไฟล์ รูปแบบของข้อมูลที่เก็บในไฟล์ มีรูปแบบดังนี้

```
name # hp # hpmax # sp # spmax # age # level # energy # status # full # thirsty #
str # qi # def # agi # dex # fit # vit # breath # Int # team # space # body # exp #   exprequired !
```

name	=	ข้อมูลชื่อของตัวละคร
hp	=	ข้อมูลพลังชีวิตปัจจุบันของตัวละคร
hpmax	=	ข้อมูลพลังชีวิตสูงสุดของตัวละคร
sp	=	ข้อมูลพลังวิญญานของตัวละคร
spmax	=	ข้อมูลพลังวิญญานสูงสุดของตัวละคร
age	=	ข้อมูลอายุของตัวละคร
level	=	ข้อมูลค่า Level ของตัวละคร
energy	=	ข้อมูลค่าพลังงาน Energy ของตัวละคร
status	=	ข้อมูลค่าสถานะของตัวละคร
full	=	ข้อมูลค่า Full ของตัวละคร

thirsty	=	ข้อมูลค่า Thirsty ของตัวละคร
str, qi, def, agi, dex, fit, vit, breath, Int	=	ค่าพารามิเตอร์ของตัวละคร
team	=	ข้อมูลบอกว่ามอนสเตอร์เป็นมอนสเตอร์ของฝ่ายไหน
space	=	ข้อมูลค่า Space ขอบเขตการฝึกฝนของตัวละคร
body	=	ข้อมูลบอกชนิดร่างกายของตัวละคร
exp	=	ข้อมูลค่าประสบการณ์ (Exp) ของตัวละคร
exprequired	=	ข้อมูลค่าประสบการณ์ที่ต้องใช้เพื่อเลื่อนระดับ Level ถัดไปของตัวละคร

เครื่องหมาย # ใช้สำหรับแยกแยะชุดของข้อมูลแต่ละค่า

เครื่องหมาย ! หมายถึงการบ่งบอกว่าจบข้อมูลของตัวละครที่ทำการบันทึก

3.4 การผสมพันธุ์มอนสเตอร์

การผสมพันธุ์มอนสเตอร์จะต้องมีการเชื่อมต่อระหว่างเครื่องฟ็อกเก็ตพีซีของผู้เล่น กับเครื่องเซิร์ฟเวอร์ โดยการจัดการการผสมพันธุ์มอนสเตอร์นั้น โปรแกรมแอปพลิเคชันที่เครื่องเซิร์ฟเวอร์จะเป็นผู้ดูแลและจัดการทั้งหมด

รูปแบบของข้อมูลในไฟล์ที่โปรแกรมแอปพลิเคชันในเครื่องเซิร์ฟเวอร์รับและส่งคืนแก่เครื่องฟ็อกเก็ตพีซีของผู้เล่นอธิบายอยู่ในหัวข้อย่อยที่ 2. “โปรโตคอลในการติดต่อและรูปแบบข้อมูลที่ใช้” ในหัวข้อที่ 3.8.2 “สถาปัตยกรรมชั้นพื้นฐาน (Low Level Architecture)”

กระบวนการ Genetic Algorithm

1. สร้างประชากรเริ่มต้น

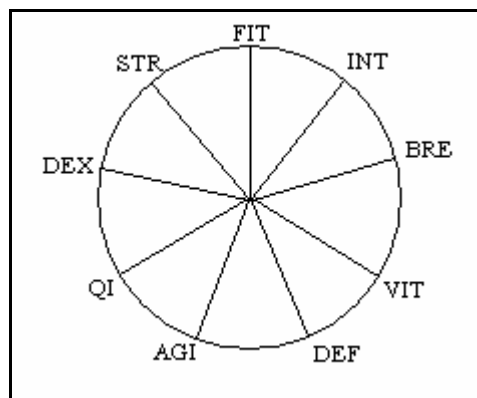
- กำหนดจำนวนประชากร 5 ตัว
- กำหนดรูปแบบ chromosome ดังนี้

$$Gene \in \{0 - 255\}$$

Chromosome ประกอบด้วย Gene 9 ตัว

2. นิยาม Fitness Function

นำค่าความสามารถของตัวละครแต่ละค่ามาเรียงในวงกลมโดยค่าที่ผลถึงกันอยู่ติดกัน กำหนดค่า Fitness Function จากพื้นที่ภายในของค่าความสามารถทั้งหมดของตัวละคร



รูปที่ 3.1 แสดงการออกแบบตำแหน่งพารามิเตอร์ที่สัมพันธ์กัน

$$\text{สูตร : } (FIT+INT)^2 + (INT+BRE)^2 + (BRE+VIT)^2 + (VIT+DEF)^2 + (DEF+AGI)^2 + (AGI+QI)^2 + (QI+DEX)^2 + (DEX+STR)^2 + (STR+FIT)^2$$

3. คำนวณ Fitness Value ของแต่ละ Chromosome
4. Selection พ่อพันธุ์ แม่พันธุ์ (Rank Selection) ขั้นตอนการเลือกจะขึ้นกับอัตราส่วนของ Fitness Value โดยที่โครโมโซมที่มีค่า Fitness Value มาก จะมีโอกาสได้รับเลือกมาก
5. ทำ GA operation
 - Crossover

Crossover rate เป็น 0.1 คือมี Crossover Point เพียงจุดเดียว

Crossover point เลือกจากการสุ่ม
 - Mutation

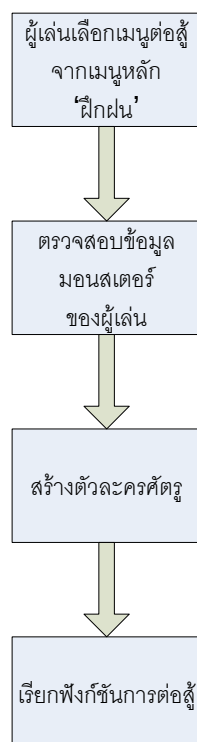
เลือก Mutation rate = 0.1
6. นำประชากรที่ได้ใหม่ไปแทนที่ประชากรชุดเดิมโดยคัดเลือกโครโมโซมที่มีค่า Fitness Value น้อยที่สุดออก
7. ทำซ้ำ ที่ step 3 ทำซ้ำประมาณ 10 รอบ

เมื่อสิ้นสุดกระบวนการจะได้ข้อมูลของมอนสเตอร์ ‘ลูก’ ซึ่งเป็นผลลัพธ์จากกระบวนการผสมพันธุ์นั่นเอง แล้วเครื่องเซิร์ฟเวอร์จะทำการส่งข้อมูลนี้กลับไปให้ผู้เล่นเก็บข้อมูลของมอนสเตอร์ลูกไว้ในเมนู ‘ห้องเก็บไข่’

3.5 การออกแบบอัลกอริทึมที่ใช้ในการต่อสู้

3.5.1 การต่อสู้ระหว่างผู้เล่นกับคอมพิวเตอร์

การต่อสู้ระหว่างผู้เล่นกับคอมพิวเตอร์สามารถทำได้โดยเลือกเมนูต่อสู้ภายในเกมบนเครื่องของผู้เล่นได้เอง

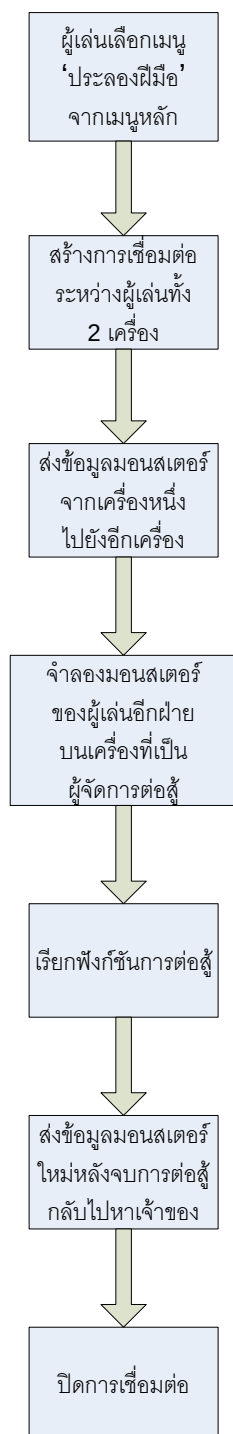


รูปที่ 3.2 แสดงการเข้าสู่การต่อสู้ระหว่างผู้เล่นกับคอมพิวเตอร์

3.5.2 การต่อสู้ระหว่างผู้เล่นกับผู้เล่นคนอื่น

การต่อสู้ระหว่างผู้เล่นกับผู้เล่นคนอื่นต้องทำการเชื่อมต่อระหว่างเครื่องของผู้เล่นทั้งสองก่อน

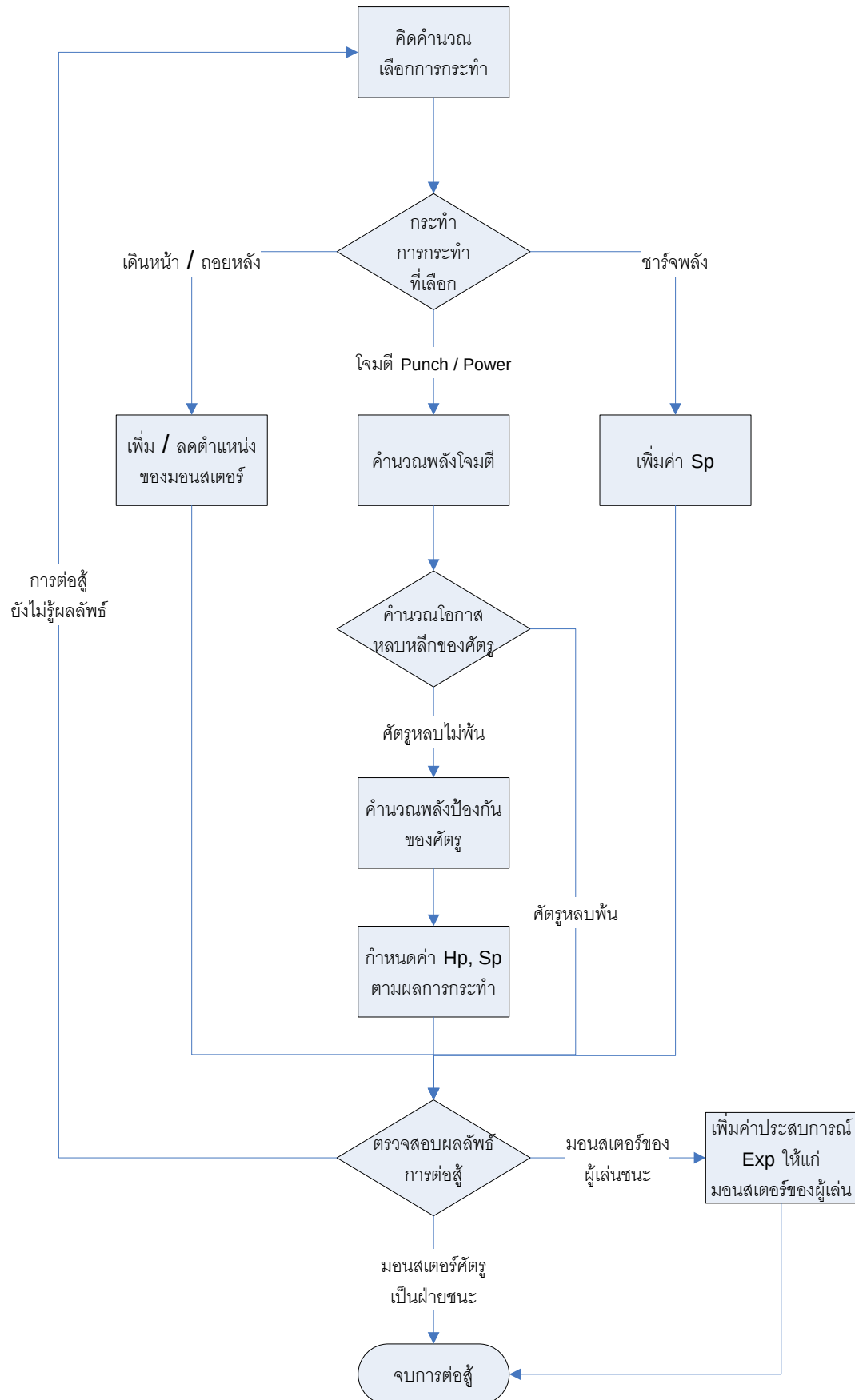
การดำเนินการต่อสู้กระทำบนเครื่องของผู้เล่นเครื่องใดเครื่องหนึ่งเท่านั้น (ผู้เล่นที่ไม่ได้เป็นผู้จัดการต่อสู้สามารถชมการต่อสู้ได้บนเครื่องของผู้เล่นอีกคนหนึ่ง) เมื่อการต่อสู้จบลงระบบก็จะส่งข้อมูลของมอนสเตอร์หลังจากต่อสู้กลับคืนไปยังเครื่องของผู้เล่นอีกฝ่ายหนึ่ง



รูปที่ 3.3 แสดงการเข้าสู่การต่อสู้ระหว่างผู้เล่นกับผู้เล่นคนอื่น

3.5.3 การทำงานของระบบต่อสู้

หลักการทำงานต่อสู้ระหว่างมอนสเตอร์ (ทั้งแบบระหว่างผู้เล่นกับคอมพิวเตอร์ และแบบระหว่างผู้เล่นด้วยกันเอง) แสดงผังแผนภาพต่อไปนี้



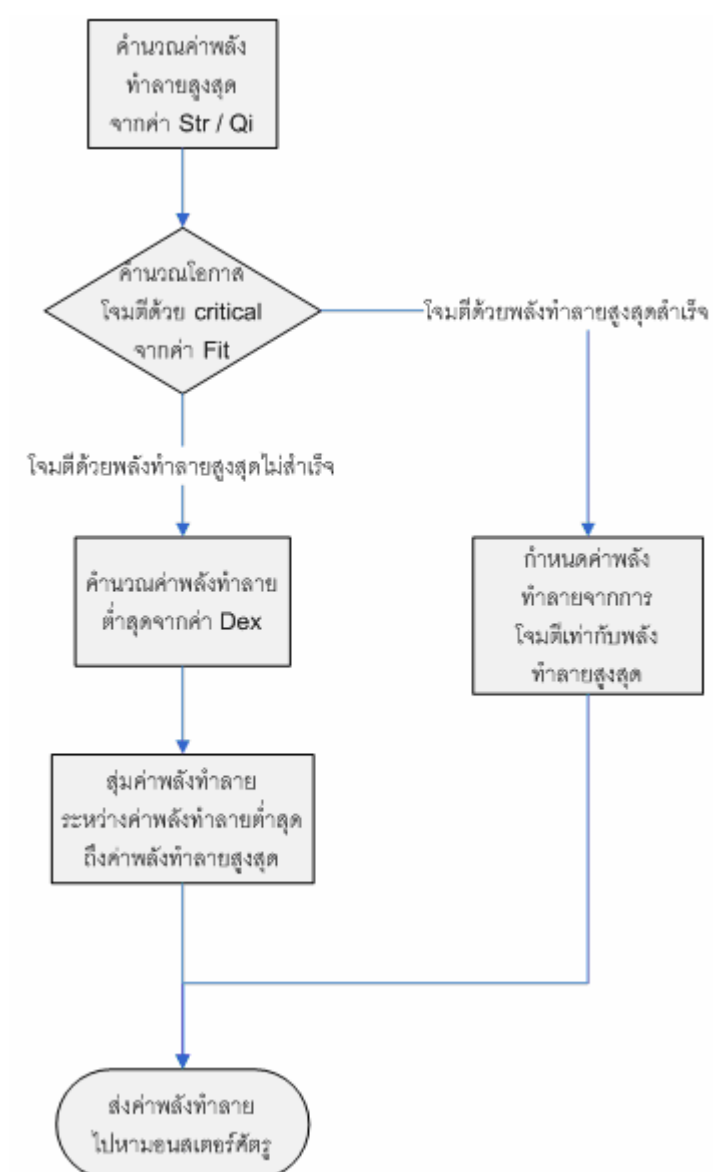
รูปที่ 3.4 แสดงระบบการจัดการการต่อสู้

3.5.3.1 การคำนวณค่าพลังโจมตี

ปัจจัยที่มีผลต่อค่าพลังโจมตีมีดังนี้

- ค่า Str - ในการโจมตีระยะใกล้
- ค่า Qi - ในการโจมตีระยะไกล
- ค่า Dex - ส่งผลต่อขอบเขตของพลังทำลาย
- ค่า Fit - ส่งผลต่อโอกาสโจมตีด้วยพลังทำลายสูงสุด

หลักการจัดการคำนวณค่าพลังทำลาย แสดงด้วยแผนภาพ ดังนี้



รูปที่ 3.5 แสดงการหาค่าพลังทำลายจากค่าพลังโจมตี

อธิบายการทำงานได้ดังนี้

1. ตรวจสอบการกระทำ (action) ที่จะกระทำ

- a. การโจมตีใกล้ (action = Punch) คำนวณค่าพลังทำลายสูงสุด (maximum_damage) ดังนี้

$$\text{damage} = \text{Str} \times 3$$

- b. การโจมตีไกล (action = Power) คำนวณค่าพลังทำลายสูงสุด (maximum_damage) ดังนี้

$$\text{damage} = \text{Qi} \times 2$$

ในขั้นตอนนี้จะได้ค่าพลังทำลายสูงสุด(maximum_damage)

2. คำนวณโอกาสได้ค่าพลังทำลายสูงสุดในการโจมตีครั้งนั้น

- a. สุ่มค่าตัวเลข (random_number) ระหว่างค่า 0 ถึง 255
- b. นำค่าพารามิเตอร์ Fit ของมอนสเตอร์มาเปรียบเทียบกับค่าตัวเลขที่ได้จากการสุ่ม (random_number) หากค่าพารามิเตอร์ Fit มีค่ามากกว่าจะถือว่าได้โอกาสโจมตีด้วยพลังทำลายสูงสุด แต่หากค่าพารามิเตอร์ Fit มีค่าน้อยกว่า ระบบจะไปคำนวณค่าพลังทำลายที่จะได้ต่อไป

จากหลักการนี้จะเห็นว่ายังมีค่า Fit มากจะยังมีโอกาสโจมตีด้วยพลังทำลายสูงสุดมากไป
ด้วย

3. คำนวณค่าพลังทำลายที่ได้จากการโจมตีครั้งนั้น

- a. ตรวจสอบโอกาสโจมตีด้วยพลังทำลายสูงสุด หากมีโอกาสจะโจมตีด้วยพลังทำลายสูงสุด
- b. ในกรณีที่ไม่มีโอกาสโจมตีด้วยพลังทำลายสูงสุด คำนวณค่าพลังทำลายต่ำสุด โดยนำค่าพารามิเตอร์ Dex มาใช้ แต่มีข้อจำกัดว่า ค่าพลังทำลายต่ำสุด (minimum_damage) มีค่าได้ไม่เกินครึ่งหนึ่งของค่าพลังทำลายสูงสุด (maximum_damage) ดังนี้

$$\text{ค่าพลังทำลายต่ำสุด (minimum_damage)} = \text{Dex} \times 2$$

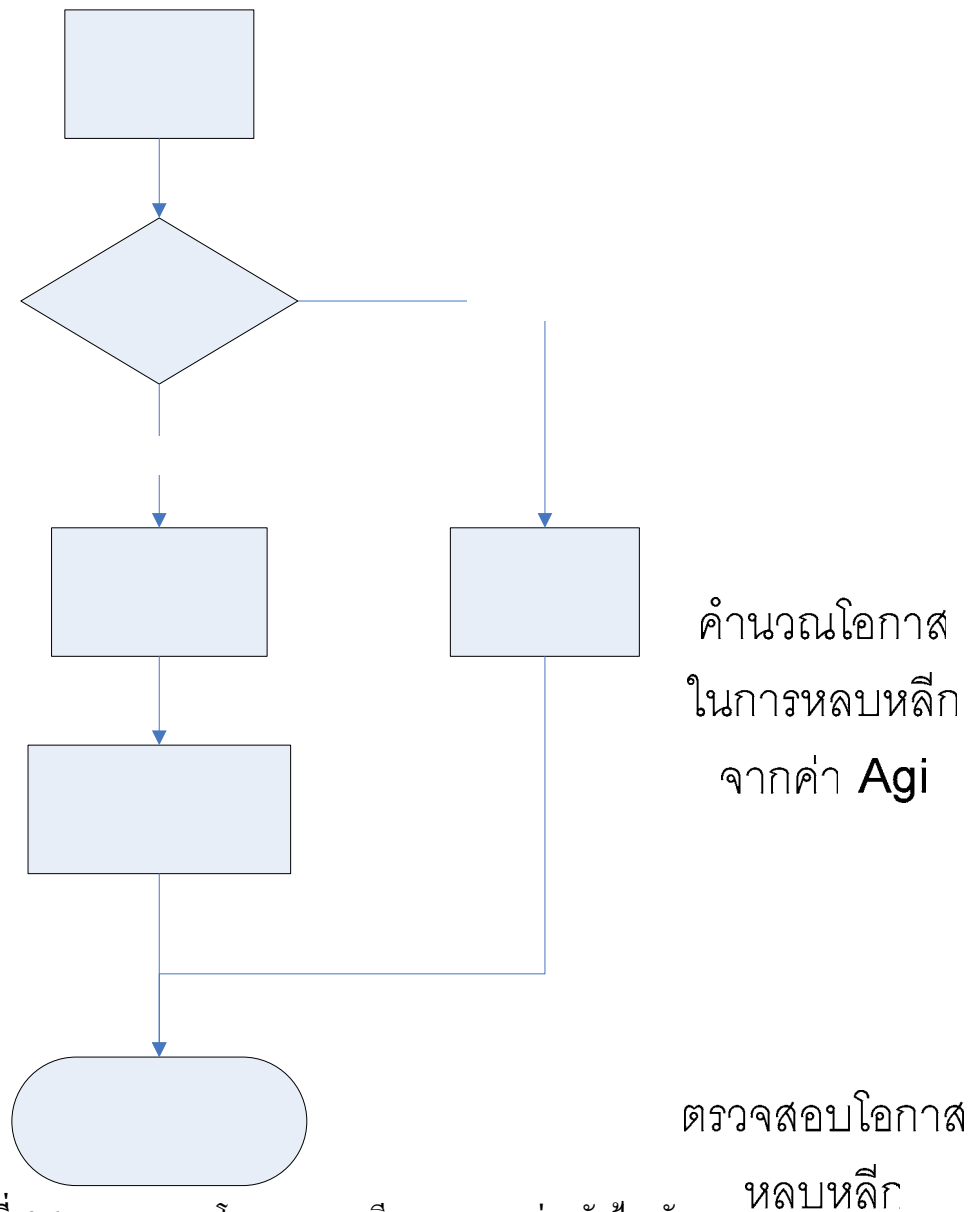
$$\text{โดยที่ } \text{minimum_damage} \leq \text{maximum_damage} / 2$$

- c. สุ่มค่าพลังทำลาย (damage) ระหว่างค่าพลังทำลายต่ำสุด (minimum_damage) จนถึง ค่าพลังทำลายสูงสุด (maximum_damage)
- d. กำหนดให้ค่าพลังทำลายในการโจมตีครั้งนั้นเท่ากับค่าพลังทำลายที่สุ่มได้ (ค่า damage)

จากหลักการนี้จะเห็นว่ายิ่งมีค่า Dex มากจะยิ่งมีขอบเขตพลังทำลายยิ่งมากตาม

3.5.3.2 การคำนวณอัตราการหลบหลีก

หลักการรับการโจมตีจากมอนสเตอร์ศัตรูแสดงในแผนภาพต่อไปนี้



รูปที่ 3.6 แสดงการหาโอกาสหลบหลีกและการหาค่าพลังป้องกัน

การคำนวณหาอัตราการหลบหลีกมีขั้นตอนดังต่อไปนี้

1. สุ่มค่าตัวเลข (random_number) ตั้งแต่เลข 0 ถึง 255
2. เปรียบเทียบค่าตัวเลขที่ได้จากการสุ่ม กับค่าพารามิเตอร์ Agi ของตัวละคร

หลบหลีกไม่สำเร็จ

- a. หากค่า Agi มากกว่าหรือเท่ากับ ถือว่าหลบหลีกได้ จะกำหนดค่าพลังทำลายจากการโจมตีของศัตรูให้เท่ากับ 0
- b. หากค่า Agi น้อยกว่า ถือว่าหลบหลีกไม่ได้ ระบบจะทำการคำนวณค่าพลังป้องกันเป็นลำดับถัดไป

3.5.3.3 การคำนวณค่าพลังป้องกัน

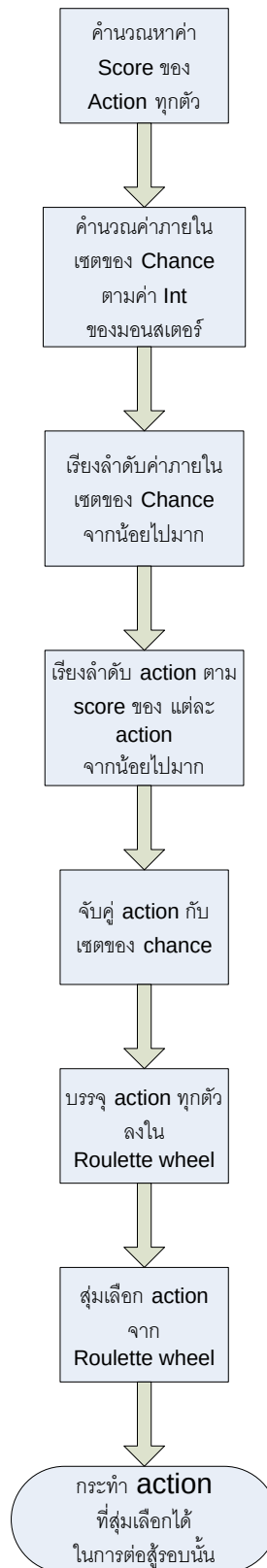
จากรูปที่ 3.6 การคำนวณค่าพลังป้องกันมีหลักการดังต่อไปนี้

1. คำนวณค่าพลังป้องกันสูงสุดจากค่าพารามิเตอร์ Def ดังนี้

$$\text{ค่าพลังป้องกันสูงสุด (maximum_def)} = (\text{ค่าพารามิเตอร์ Def}) \times 3$$
2. สุ่มตัวเลขหาค่าพลังป้องกัน (Defense_value) จากค่าระหว่าง 0 ถึง พลังป้องกันสูงสุด (maximum_def)
3. กำหนดค่าพลังป้องกันของตัวมอนสเตอร์ต่อการถูกโจมตีครั้งนั้นเท่ากับค่าตัวเลขที่สุ่มได้

3.5.3.4 การเลือกการกระทำในการต่อสู้

หลักการคิดคำนวณของมอนสเตอร์ในการต่อสู้ แสดงในแผนภาพต่อไปนี้



รูปที่ 3.7 แสดงการคำนวณเลือก action ที่กระทำในการต่อสู้

รายละเอียดของการออกแบบมีดังต่อไปนี้

1. ในฉากต่อสู้มอนสเตอร์มีเซตของการกระทำ (action) ดังนี้

Action (A) = {MoveForward, MoveBackward, Punch, Power, Charge}

โดยรายละเอียดได้กล่าวถึงในตารางที่ 3.2 แล้ว

ในฉากต่อสู้มีสิ่งแวดล้อม (Environment) ดังนี้

Environment (E) = {HPe, HPme, SPme, Time, De, Str, Qi, Agi, Def}

HPe = ค่าพลังชีวิตในขณะปัจจุบันของมอนสเตอร์ศัตรู

HPme = ค่าพลังชีวิตในขณะปัจจุบันของมอนสเตอร์

SPme = ค่าพลังวิญญานในขณะปัจจุบันของมอนสเตอร์

Time = ค่าเวลาที่เหลืออยู่ในฉากต่อสู้

De = ระยะห่างระหว่างตัวมอนสเตอร์ กับตัวมอนสเตอร์ศัตรู แบ่งเป็น 3 ระยะ
ดังนี้

■ Near = ระยะใกล้ สามารถโจมตีระยะใกล้ถึง

■ Medium = ระยะกลาง

■ Far = ระยะไกลมาก

Str, Qi, Agi, Def = ค่าพารามิเตอร์ที่ใช้ในการต่อสู้ของมอนสเตอร์ (มี

อธิบายรายละเอียดของแต่ละตัวในตารางที่ 3.1)

ในฉากต่อสู้ตัวละครมอนสเตอร์มีเป้าหมาย (Goal) ดังนี้

Goal (G) = {HPe, HPme, SPme, AtkOpp, DefOpp}

HPe = ลดค่าพลังชีวิต (HP) ของศัตรู

HPme = ปกป้องรักษาพลังชีวิต (HP) ของตัวเอง

SPme = ปกป้องรักษาพลังวิญญาน (SP) ของตัวเอง

AtkOpp = สร้างโอกาสที่จะได้โจมตีคู่ต่อสู้

DefOpp = หลีกเลียงสถานการณ์ที่จะถูกโจมตี

การหาค่าคะแนน (Score) ของการกระทำ action ใดๆ ในสถานการณ์ขณะนั้นกระทำได้
โดยนำฟังก์ชันที่ช่วยคำนวณผลการกระทำ action นั้นๆ มาใช้

Action \ Goal	HPe	HPme	SPme	AtkOpp	DefOpp	SUM
MoveForward	Func1(a1)	Func2(a1)	Func3(a1)	Func4(a1)	Func5(a1)	Sum(a1)
MoveBackward	Func1(a2)	Func2(a2)	Func3(a2)	Func4(a2)	Func5(a2)	Sum(a2)
Punch	Func1(a3)	Func2(a3)	Func3(a3)	Func4(a3)	Func5(a3)	Sum(a3)
Power	Func1(a4)	Func2(a4)	Func3(a4)	Func4(a4)	Func5(a4)	Sum(a4)
ChargeSP	Func1(a5)	Func2(a5)	Func3(a5)	Func4(a5)	Func5(a5)	Sum(a5)

ตารางที่ 3.4 แสดงการหาค่า Score ของแต่ละ Action

จากตารางที่ 3.4 ฟังก์ชัน Func(action) ทำการนำ action ประเภะนั้นมาคิดคำนวณกับสิ่งแวดล้อม (Environment) ในปัจจุบันว่าการกระทำ action นั้นๆ สนับสนุนให้ Goal มีความสำเร็จมากขึ้นเพียงใด

Func1(Action) ฟังก์ชันสำหรับส่งค่า Score ที่ได้จากการกระทำ action ใดๆ แล้วมีผลต่อ Goal::HPe มากน้อยแค่ไหน กลับมา

Func2(Action) ฟังก์ชันสำหรับส่งค่า Score ที่ได้จากการกระทำ action ใดๆ แล้วมีผลต่อ Goal::HPme มากน้อยแค่ไหน กลับมา

Func3(Action) ฟังก์ชันสำหรับส่งค่า Score ที่ได้จากการกระทำ action ใดๆ แล้วมีผลต่อ Goal::SPme มากน้อยแค่ไหน กลับมา

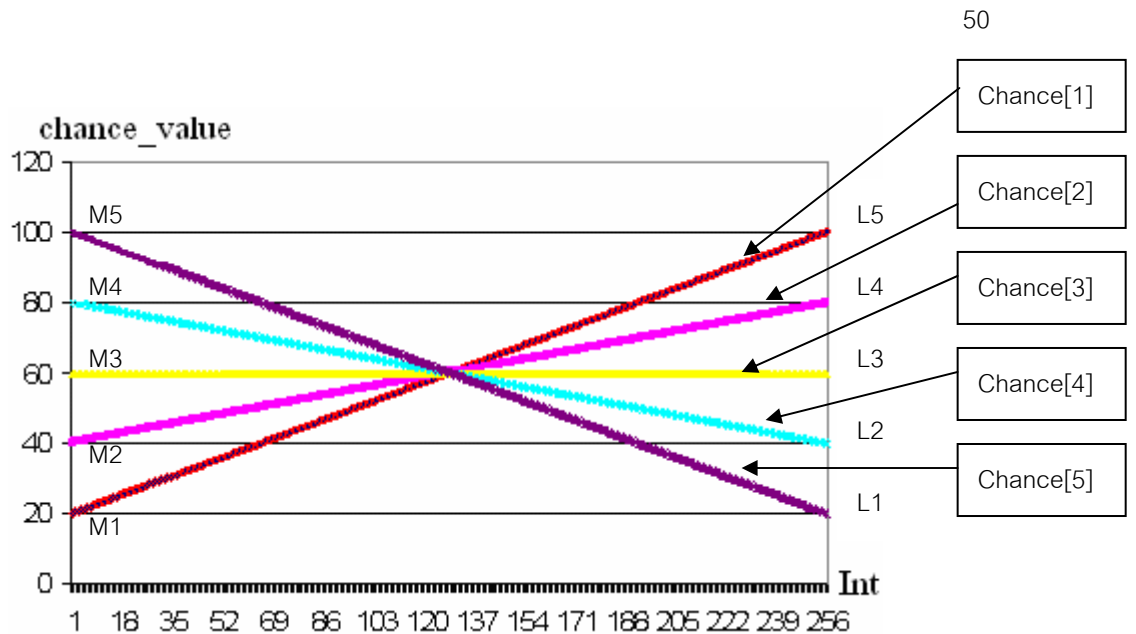
Func4(Action) ฟังก์ชันสำหรับส่งค่า Score ที่ได้จากการกระทำ action ใดๆ แล้วมีผลต่อ Goal::AtkOpp มากน้อยแค่ไหน กลับมา

Func5(Action) ฟังก์ชันสำหรับส่งค่า Score ที่ได้จากการกระทำ action ใดๆ แล้วมีผลต่อ Goal::DefOpp มากน้อยแค่ไหน กลับมา

เมื่อได้ค่า score ของ action ใดๆ จากการคำนวณในทุก Goal แล้วนำมาหาค่ารวมเฉพาะ action นั้นๆ action ใดที่มีผลรวมของ score มากที่สุดถือว่าเป็น action ที่ดีที่สุดในขณะนั้น

2. เซ็ตของ Chance คือ เซ็ตที่เก็บค่าโอกาสที่จะกระทำ action ที่ดีมาก – น้อย ต่างกันไป โดยขึ้นกับค่าพารามิเตอร์ความฉลาด (Int) ของตัวมอนสเตอร์ ในที่นี้มีประกอบด้วย

{Chance[1], Chance[2], Chance[3], Chance[4], Chance[5]}



รูปที่ 3.8 แสดงค่าเฉลี่ยของค่า chance

จากรูปที่ 3.8 ค่า Chance[1] เก็บค่า Chance ที่มีประสิทธิภาพสูงสุด ซึ่งจะมีค่าเพิ่มขึ้นเมื่อค่าพารามิเตอร์ Int เพิ่มขึ้น โดยที่ประสิทธิภาพของค่า Chance สามารถเรียงลำดับได้ดังนี้

$$\text{Chance}[1] > \text{Chance}[2] > \text{Chance}[3] > \text{Chance}[4] > \text{Chance}[5]$$

โดยวัดตามค่าที่เพิ่มขึ้นเมื่อค่าพารามิเตอร์ Int ของมอนสเตอร์เพิ่มมากขึ้น ในระบบเกมนี้กำหนดให้ Chance[1] มีหน้าที่เก็บ action ที่ดีที่สุด และ Chance[2] มีหน้าที่เก็บ action ที่ดีรองลงมาเป็นอันดับสอง และไล่ลงมาเรื่อยๆ จนถึง Chance[5] มีหน้าที่เก็บ action ที่แย่ที่สุด

ในสภาวะการณ์ตอนนั้น Chance[] ใดมีค่ามากที่สุดจะขึ้นกับค่า Int ตามสมการหาความชันต่อไปนี้

$$\text{Chance}[1] = (((L5 - M1) / 255) * \text{Int}) + M1$$

$$\text{Chance}[2] = (((L4 - M2) / 255) * \text{Int}) + M2$$

$$\text{Chance}[3] = (((L3 - M3) / 255) * \text{Int}) + M3$$

$$\text{Chance}[4] = (((L2 - M4) / 255) * \text{Int}) + M4$$

$$\text{Chance}[5] = (((L1 - M5) / 255) * \text{Int}) + M5$$

3. เมื่อได้ชุดข้อมูล Score รวมของแต่ละ Action และค่า Chance[] แล้ว ระบบทำการจับคู่ระหว่าง Action กับ Chance[] ที่เหมาะสม ดังนี้

Chance[1] จับคู่กับ Action ที่มีผลรวม Score มากเป็นอันดับ 1

Chance[2] จับคู่กับ Action ที่มีผลรวม Score มากเป็นอันดับ 2

Chance[3] จับคู่กับ Action ที่มีผลรวม Score มากเป็นอันดับ 3

Chance[4] จับคู่กับ Action ที่มีผลรวม Score มากเป็นอันดับ 4

Chance[5] จับคู่กับ Action ที่มีผลรวม Score มากเป็นอันดับ 5 หรือน้อยที่สุด

4. นำค่าที่เก็บอยู่ใน Chance[] ไปใส่ใน Roulette Wheel ขนาด 15 ช่อง ดังนี้

- ช่อง 0 ถึง 4 จำนวน 5 ช่อง บรรจุ Action ภายใน Chance[] ที่มีค่า Chance มากที่สุด
- ช่อง 5 ถึง 8 จำนวน 4 ช่อง บรรจุ Action ภายใน Chance[] ที่มีค่า Chance มากเป็นลำดับที่ 2
- ช่อง 9 ถึง 11 จำนวน 3 ช่อง บรรจุ Action ภายใน Chance[] ที่มีค่า Chance มากเป็นลำดับที่ 3
- ช่อง 12 ถึง 13 จำนวน 2 ช่อง บรรจุ Action ภายใน Chance[] ที่มีค่า Chance มากเป็นลำดับที่ 4
- ช่อง 14 จำนวน 1 ช่อง บรรจุ Action ภายใน Chance[] ที่มีค่า Chance น้อยที่สุด

5. สุ่มเลือก Action ที่อยู่ภายใน Roulette Wheel เป็น Action ที่กระทำการตัดสินใจเลือก Action ที่จะใช้ในการต่อสู้รอบนั้นๆ

6. สรุป : หากมอนสเตอร์มีค่าพารามิเตอร์ความฉลาด (ค่า Int) สูง จะส่งผลให้ค่า Chance[1] ยิ่งมีค่ามาก ซึ่งส่งผลให้ Action ที่ดีที่สุด (มี Score สูงที่สุด) ถูกบรรจุใน Roulette Wheel จำนวน 5 ช่อง (มีอยู่ในที่สำหรับสุ่มเลือกเป็นจำนวนมาก) ส่งผลให้โอกาสที่มอนสเตอร์จะเลือก Action ที่ดีที่สุดมากขึ้นไปด้วย

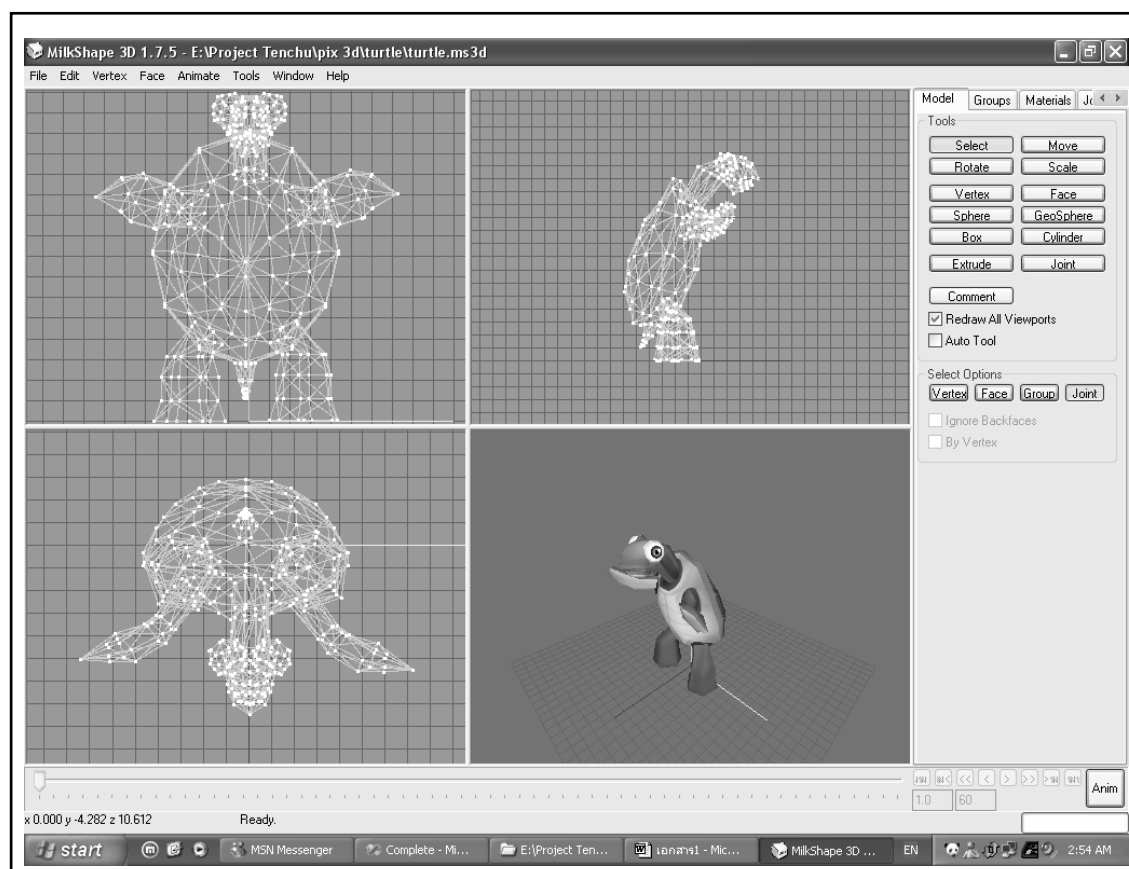
ในขณะที่ ค่า Int มาก จะส่งผลให้ ค่า Chance[5] มีค่าต่ำ ซึ่งส่งผลให้ Action ที่แย่ที่สุด (มีค่า Score น้อยที่สุด) มีโอกาสถูกเลือกน้อยมาก

3.6 การออกแบบกราฟิกสามมิติ

3.6.1 การออกแบบส่วนโมเดล 3 มิติ

การสร้างโมเดล 3 มิติภายในเกมแบ่งออกเป็น 4 ส่วนดังนี้

1. ส่วนในการสร้างโมเดล 3 มิติใช้โปรแกรม Milkshape 3D ในการพัฒนา

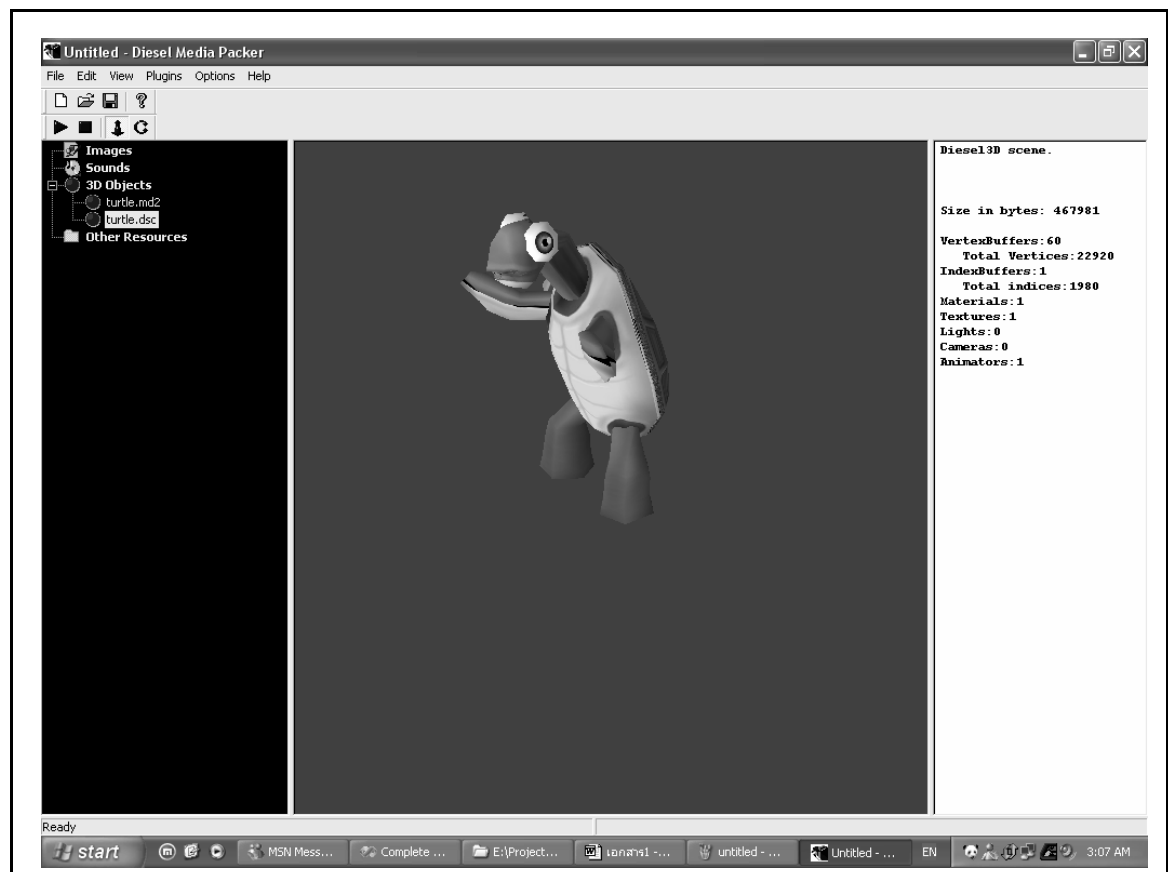


รูปที่ 3.9 โปรแกรม Milkshape3D

สร้างรูปโมเดลโดยแบ่งโมเดลออกเป็นกลุ่มย่อยหลายๆกลุ่ม จากนั้นสร้างโครงตามแนวทางดังต่อไปนี้

- กำหนดจุด 3 จุดเพื่อใช้เป็นพื้นผิวในหนึ่งกลุ่มย่อย แล้วทำการสร้างพื้นผิวจนครบตัวโมเดล
- ใช้ฟังก์ชันพื้นฐานสร้างเป็น ทรงกลม ทรงสี่เหลี่ยม ทรงกระบอก แล้วทำการย้าย ขยาย จุด จนได้โครงที่ต้องการ
- หาตัวละครที่เป็น Open Source มาทำการแก้ไข อาจใช้ทั้งหมด หรือตัดมาเพียงบางส่วนแล้วมาประกอบกับส่วนอื่นๆ

2. ส่วนในการแต่งพื้นผิว จะอยู่ในแท็บ Material หลังจาก assign โมเดลให้กับ Texture แล้ว สามารถกำหนดตำแหน่งอย่างละเอียดได้ด้วย Texture Coordinate Editor
3. ส่วนในการทำภาพเคลื่อนไหว จำเป็นต้องสร้าง ข้อต่อ (Joint) ให้กับโมเดลก่อนพร้อมทั้ง Assign โมเดลแต่ละส่วนให้กับข้อต่อนั้นๆ เมื่อเข้าสู่ Animation Mode ให้ทำการขยับ และ หมุน ข้อต่อ ตามแต่ต้องการกระทำนั้นๆ
4. การแปลงไฟล์โมเดล 3 มิติให้อยู่ในรูปแบบที่นำไปใช้งานในเกมได้
 ให้ทำการ Export ไฟล์เป็นสกุล .MD2 แล้วนำไฟล์ดังกล่าวไปปรับแก้ในโปรแกรม 'DieselMediaPacker' เมื่อ ได้โมเดลที่ต้องการแล้วให้ Export Item ไปเป็นไฟล์สกุล .DSC (Diesel3D Scene Converter)



รูปที่ 3.10 โปรแกรม Diesel Media Packer

3.6.2 วิธีการนำโมเดล 3 มิติไปใช้งานบนคอมพิวเตอร์แบบพกพา

- สร้างหน้าต่างของโปรแกรมขึ้นมาโดยใช้ คลาส IDieselApplication โดยกำหนดหลายละเอียดต่างๆ ให้นักต่างและเริ่มต้นทำงานด้วย
- สร้างหน้าต่างควบคุมการแสดงผลภาพโดยใช้คลาส CDieselD3DDevice และกำหนดตัวแปรในการแสดงผลเช่นกำหนดค่า z-buffer
- ทำการสร้าง path ไปยัง file ของ animation แล้วทำการโหลดข้อมูลไว้ในคลาส CDiesel3DScene
- เปลี่ยนแปลงการแสดงผลของ animation ไปตาม frametime ของ animation ทำให้ภาพกราฟิกแสดงไปเรื่อยๆ
- ทำการลบหน้าจอเดิมออกจาก buffer แล้วทำการเขียนภาพใหม่ลงไปบนหน้าจอ

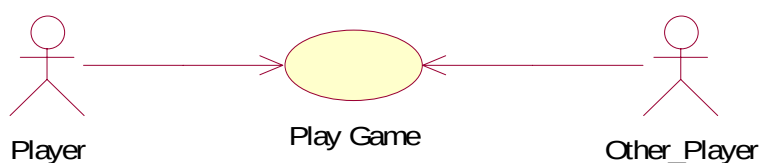
3.7 การออกแบบเกม

3.7.1 Use case diagram

หัวข้อนี้แสดง Use case diagram ในมุมมอง 2 ระดับ คือ

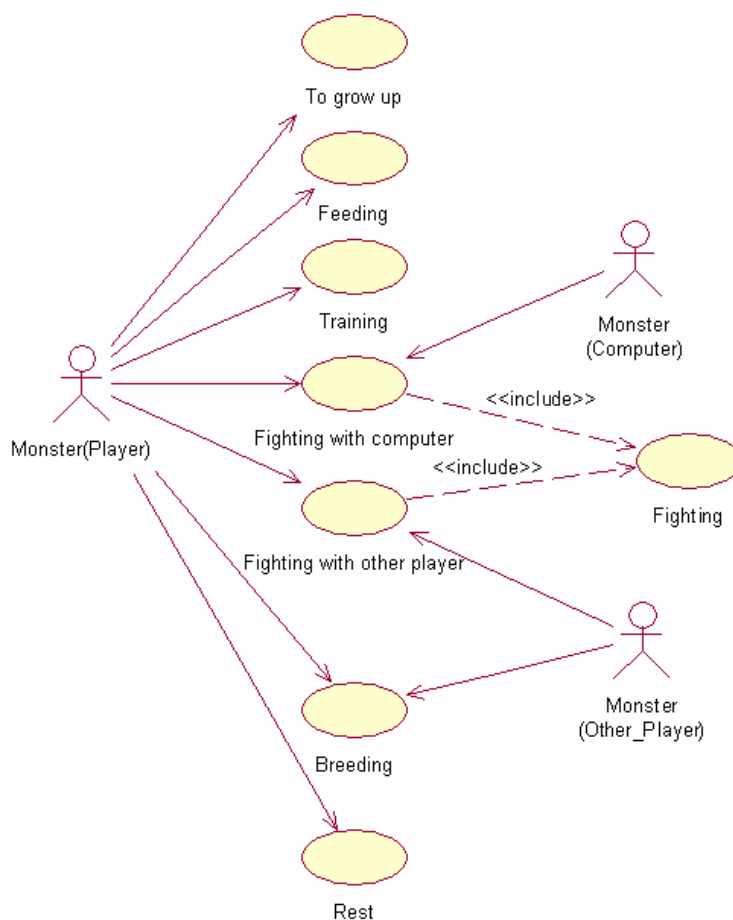
- 1) มุมมองระดับผู้เล่น แสดงสิ่งที่ผู้เล่นสามารถกระทำกับระบบได้
- 2) มุมมองระดับภายในเกม แสดงสิ่งที่ตัวละครอนสเตอร์ภายในเกมสามารถกระทำกับระบบได้

3.7.1.1 Use case diagram ในมุมมองผู้เล่น



รูปที่ 3.11 แสดง Use case diagram ของระบบ

3.7.1.2 Use case diagram ในมุมมองระดับภายในเกม



รูปที่ 3.12 แสดง Use case diagram ของเกม

3.7.1.3 Use Case Description

หัวข้อนี้อธิบายรายละเอียดของแต่ละ Use case จาก Use case diagram

Use Case :

Play Game

Short Description :

การเล่นเกม

Actor :

Player, Other_Player

Pre – Conditions :

1. ผู้เล่นมีอุปกรณ์ฟ็อกเก็ตพีซี ที่มีระบบสื่อสารไร้สาย
2. ผู้เล่นมีแอปพลิเคชันเกมเลี่ยมอนสเตอร์ 3 มิติ

Post – Conditions :

1. ผู้เล่นสามารถเล่นเกมเลี้ยวมอนสเตอร์ 3 มิตินี้ได้
2. ผู้เล่นสามารถเล่นเกมเลี้ยวมอนสเตอร์ 3 มิตินี้ร่วมกับผู้เล่นคนอื่นได้

Main Flow :

1. ผู้เล่นลงโปรแกรมแอปพลิเคชันเลี้ยวมอนสเตอร์ 3 มิตินี้
2. ผู้เล่นเล่นเกม โดยเลือกเมนูต่างๆ ได้ตามใจ
3. ในกรณีที่ต้องการเล่นร่วมกับผู้เล่นคนอื่นต้องทำการเปิดระบบติดต่อไร้สาย

Alternate Flow(s) :

-

Exception Flow(s) :

-

Use Case :

To grow up

Short Description :

การเจริญเติบโตของตัวละครมอนสเตอร์

Actor :

Monster(Player)

Pre – Conditions :

ตัวละครมีค่าประสบการณ์และระดับความเก่งถึงระดับหนึ่ง

Post – Conditions :

ตัวละครเปลี่ยนร่างตามสายชนิดของตัวละครที่ได้รับการฝึกฝนมา

Main Flow :

1. ผู้เล่นฝึกฝนเพิ่มค่าประสบการณ์จนระดับความเก่ง (Level) เพิ่มขึ้น
2. ระบบตรวจสอบสถานะและค่าพารามิเตอร์ของตัวละคร
3. ระบบเปลี่ยนร่างตัวละครตามชนิดของตัวละคร

Alternate Flow(s) :

-

Exception Flow(s) :

-

Use Case :

Feeding

Short Description :

การกินอาหารของมอนสเตอร์

Actor :

Monster(Player)

Pre – Conditions :

-

Post – Conditions :

ระดับค่า Full หรือระดับค่า Thirsty ของตัวละครเพิ่มขึ้น

Main Flow :

1. ผู้เล่นเลือกเมนูให้อาหารจากหน้าจอหลัก
2. ผู้เล่นเลือกชนิดของอาหารที่ต้องการให้มอนสเตอร์กิน
3. ในกรณีที่ผู้เล่นเลือกชนิดเป็นอาหารระดับค่า Full ของตัวละครเพิ่มขึ้น
4. ในกรณีที่ผู้เล่นเลือกชนิดเป็นเครื่องดื่มระดับค่า Thirsty ของตัวละครเพิ่มขึ้น

Alternate Flow(s) :

-

Exception Flow(s) :

-

Use Case :

Training

Short Description :

การฝึกฝนของตัวละคร

Actor :

Monster(Player)

Pre – Conditions :

ตัวละครมีค่าพลังงาน (Energy) เพียงพอกับเมนูฝึกฝนนั้นๆ

Post – Conditions :

1. ค่าประสบการณ์ (Exp) ของตัวละครเพิ่มขึ้น

2. ค่าพารามิเตอร์ความเก่งของตัวละครเพิ่มขึ้นตามเมนูการฝึกฝนที่ผู้เล่นเลือก
3. ค่าพลังงาน (Energy) ของตัวละครลดลงตามเมนูการฝึกฝนที่ผู้เล่นเลือก

Main Flow :

1. ผู้เล่นเลือกเมนูฝึกฝนจากหน้าจอหลัก
2. ผู้เล่นเลือกชนิดประเภทการฝึกฝน
3. ระบบตรวจสอบสถานะค่าพลังงาน (Energy) ของตัวละคร [E1]
4. ระบบเพิ่มค่าพารามิเตอร์ตามประเภทการฝึกที่ผู้เล่นเลือก [E2]
5. ระบบเพิ่มค่าประสบการณ์ให้แก่ตัวละครของผู้เล่น [A1]
6. ระบบลดค่าพลังงาน (Energy) ของตัวละคร

Alternate Flow(s) :

[A1] ค่าประสบการณ์ของตัวละครสามารถเพิ่มได้จากการต่อสู้ด้วย

Exception Flow(s) :

[E1] ตัวละครมีค่าพลังงาน (Energy) ไม่เพียงพอต่อการฝึกฝน

[E2] ตัวละครมีค่าพารามิเตอร์ Space เต็มแล้ว ค่าพารามิเตอร์จึงไม่สามารถเพิ่มได้

Use Case :

Fighting with computer

Short Description :

การต่อสู้ระหว่างตัวละครมอนสเตอร์ของผู้เล่นกับตัวละครมอนสเตอร์ศัตรูจากระบบ

Actor :

Monster(Player), Monster(Computer)

Pre – Conditions :

ตัวละครของผู้เล่นมีค่าพลังชีวิต (Hp) มากกว่า 0

Post – Conditions :

ตัวละครที่เป็นผู้ชนะการต่อสู้ได้รับค่าประสบการณ์หลังจบการต่อสู้

Main Flow :

1. ผู้เล่นเลือกเมนู ‘ฝึกฝนต่อสู้’ ซึ่งอยู่ภายในเมนู ‘ฝึกฝน’ จากหน้าจอหลัก
2. ระบบตรวจสอบสถานะของตัวละคร [E1]
3. ระบบสร้างตัวละครศัตรูให้ตรงตามระดับของตัวละครของผู้เล่น
4. ระบบเข้าสู่การต่อสู้
5. ระบบตรวจสอบผลการต่อสู้

6. ระบบเพิ่มค่าประสบการณ์ให้แก่ตัวละครของผู้เล่นในกรณีที่ต่อสู้ชนะ [A1]

Alternate Flow(s) :

[A1] ค่าประสบการณ์เพิ่มได้อีกทางโดยการฝึกฝนตัวละคร

Exception Flow(s) :

[E1] ตัวละครมีพลังชีวิต (Hp) เท่ากับ 0

Use Case :

Fighting with other player

Short Description :

การต่อสู้ระหว่างตัวละครมอนสเตอร์ของผู้เล่นกับตัวละครมอนสเตอร์ของผู้เล่นคนอื่น

Actor :

Monster(Player), Monster(Player)

Pre – Conditions :

ตัวละครของผู้เล่นทั้งคู่มีค่าพลังชีวิต (Hp) มากกว่า 0

Post – Conditions :

ตัวละครที่เป็นผู้ชนะการต่อสู้ได้รับค่าประสบการณ์หลังจบการต่อสู้

Main Flow :

1. ผู้เล่นทั้งสองเปิดการเชื่อมต่อไร้สายที่อุปกรณ์ฟ็อกเก็ตพีซี
2. ผู้เล่นทั้งสองเลือกเมนู ‘ต่อสู้’ จากหน้าจอหลัก
3. ผู้เล่นทั้งสองเลือกการเชื่อมต่อระหว่างผู้เล่น
4. ระบบตรวจสอบสถานะของตัวละคร [E1]
5. ระบบเข้าสู่การต่อสู้
6. ระบบตรวจสอบผลการต่อสู้
7. ระบบเพิ่มค่าประสบการณ์ให้แก่ตัวละครของผู้เล่นที่ต่อสู้ชนะ [A1]
8. ระบบปิดการเชื่อมต่อระหว่างผู้เล่น

Alternate Flow(s) :

[A1] ค่าประสบการณ์เพิ่มได้อีกทางโดยการฝึกฝนตัวละคร

Exception Flow(s) :

[E1] ตัวละครของผู้เล่นคนใดคนหนึ่งมีพลังชีวิต (Hp) เท่ากับ 0

Use Case :

Breeding

Short Description :

การผสมพันธุ์ระหว่างมอนสเตอร์ของผู้เล่น ซึ่งระบบรองรับการเชื่อมต่อจากผู้เล่นได้ไม่เกิน 5 เครื่อง และไข่มอนสเตอร์ที่ได้จากการผสมพันธุ์จะถูกส่งไปให้กับผู้เล่นที่ทำการเชื่อมต่อกับเครื่องเซิร์ฟเวอร์เป็นคนแรก

Actor :

Monster(Player), Monster(Other_Player)

Pre – Conditions :

-

Post – Conditions :

ไข่ของมอนสเตอร์ที่ได้จากการผสมพันธุ์

Main Flow :

1. ผู้เล่นทุกคนเลือกเมนู ‘ผสมพันธุ์’ จากหน้าจอหลัก
2. ระบบทำการเชื่อมต่อระหว่างเครื่องฟ็อกเก็ตพีซีของผู้เล่นทุกคนกับเครื่องเซิร์ฟเวอร์ซึ่งเป็นเครื่องคอมพิวเตอร์
3. ระบบบันทึกข้อมูลมอนสเตอร์ของผู้เล่นทุกคน
4. ระบบทำการผสมพันธุ์มอนสเตอร์ทุกตัว จนได้ไข่มอนสเตอร์ที่บรรจุลูกมอนสเตอร์ที่ได้จากการผสมพันธุ์
5. ระบบเลือกผู้เล่นที่เชื่อมต่อกับเครื่องเซิร์ฟเวอร์เป็นคนแรกให้เป็นผู้ได้รับไข่มอนสเตอร์ โดยข้อมูลไข่มอนสเตอร์ใหม่นี้จะทับข้อมูลไข่มอนสเตอร์ที่เครื่องผู้เล่นคนนั้นมีอยู่ก่อนหน้านี้
6. ระบบส่งข้อมูลไข่มอนสเตอร์ไปให้แก่ผู้เล่นที่เลือกให้เป็นผู้ได้รับไข่
7. ระบบปิดการเชื่อมต่อระหว่างเครื่องเซิร์ฟเวอร์กับเครื่องของผู้เล่นทุกคน

Alternate Flow(s) :

-

Exception Flow(s) :

-

Use Case :

Rest

Short Description :

มอนสเตอร์ทำการพักผ่อน

Actor :

Monster(Player)

Pre – Conditions :

-

Post – Conditions :

ค่าพลังชีวิต (Hp) , ค่าพลังวิญญาน (Sp) และค่าพลังงาน (Energy) ของมอนสเตอร์ค่อยๆ
ฟื้นคืนตลอดเวลาที่ทำการพักผ่อน

Main Flow :

1. ผู้เล่นเลือกเมนู ‘พักผ่อน’ จากหน้าจอหลัก
2. ระบบเพิ่มค่าพารามิเตอร์ต่างๆ ของตัวละคร ได้แก่ ค่าพลังชีวิต (Hp) , ค่าพลังวิญญาน (Sp) และค่าพลังงาน (Energy)
3. ระบบออกจากโหมดพักผ่อนเมื่อผู้เล่นกดปุ่มใดๆ บนหน้าจอ

Alternate Flow(s) :

-

Exception Flow(s) :

-

3.7.2 Class diagram

หัวข้อนี้แสดง Class diagram ของระบบ ซึ่งประกอบไปด้วย Class diagram จาก 2 ระดับ
ดังนี้

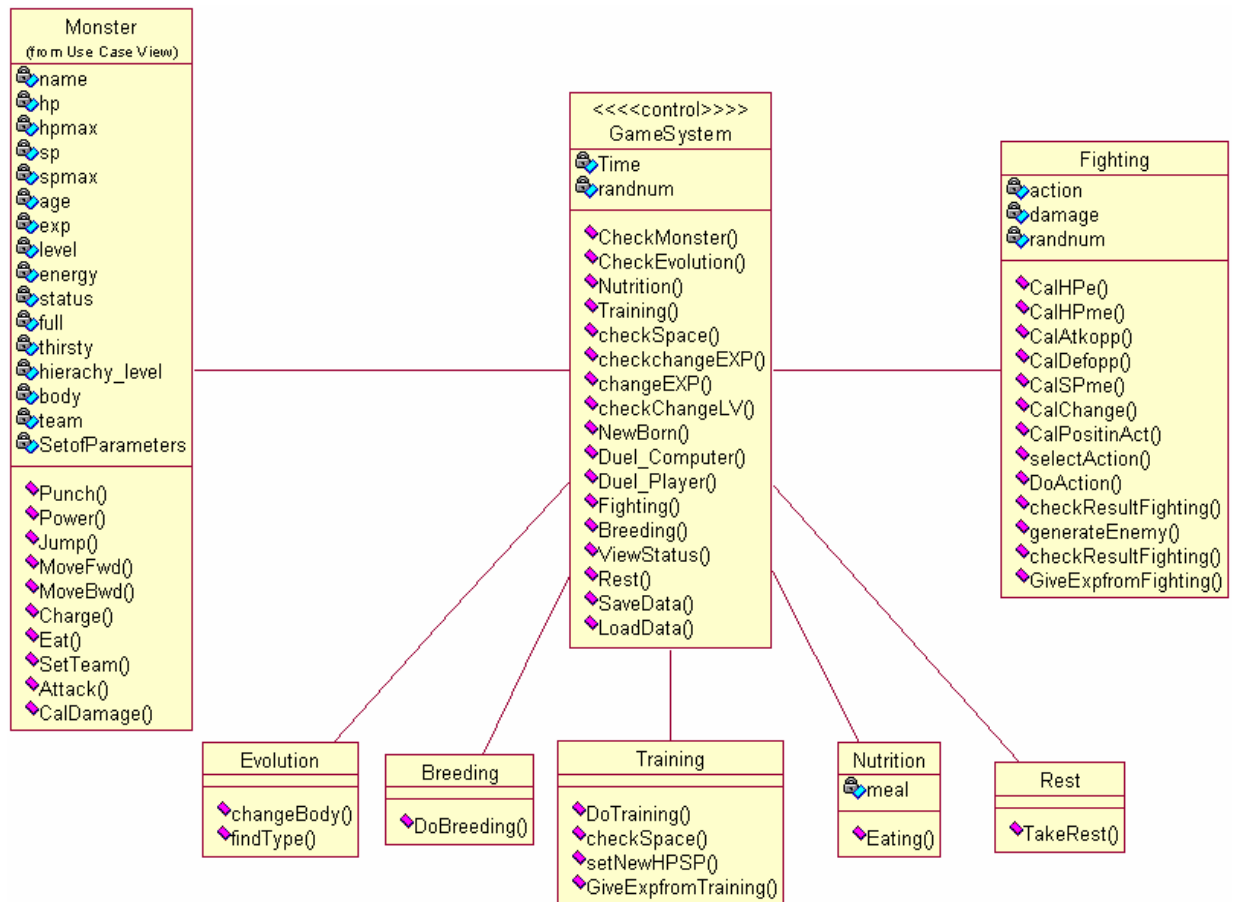
- 1) Class diagram ในมุมมองภายในเกม ระดับการวิเคราะห์ (Analysis)
- 2) Class diagram ในมุมมองภายในเกม ระดับการวิเคราะห์ (Design)

Class diagram ในระดับการวิเคราะห์ระบบ (Analysis) เป็นไดอะแกรมที่ได้จากการ
วิเคราะห์ระบบ ใช้เพื่อจำลองความเข้าใจในระบบ และใช้เป็นแหล่งข้อมูล (Resource)
ประกอบการออกแบบ (Design) ระบบในระดับถัดไป

Class diagram ในระดับการออกแบบระบบ (Design) เป็นไดอะแกรมที่ได้จากการ
ออกแบบระบบที่นำไปใช้จริง โดยนำข้อมูลจาก Class diagram ที่ได้จากการวิเคราะห์ระบบ

ไดอะแกรมในระดับการออกแบบมีความแตกต่างจากไดอะแกรมในระดับการวิเคราะห์
เนื่องจากความเหมาะสมในการนำไปใช้พัฒนาระบบจริง แต่ยังอ้างอิงข้อมูลที่ได้จากขั้นตอน
วิเคราะห์ระบบ

1. Class diagram ในมุมมองภายในเกม ระดับการวิเคราะห์ (Analysis)



รูปที่ 3.13 แสดงคลาสไดอะแกรมของเกมในระดับการวิเคราะห์ระบบ

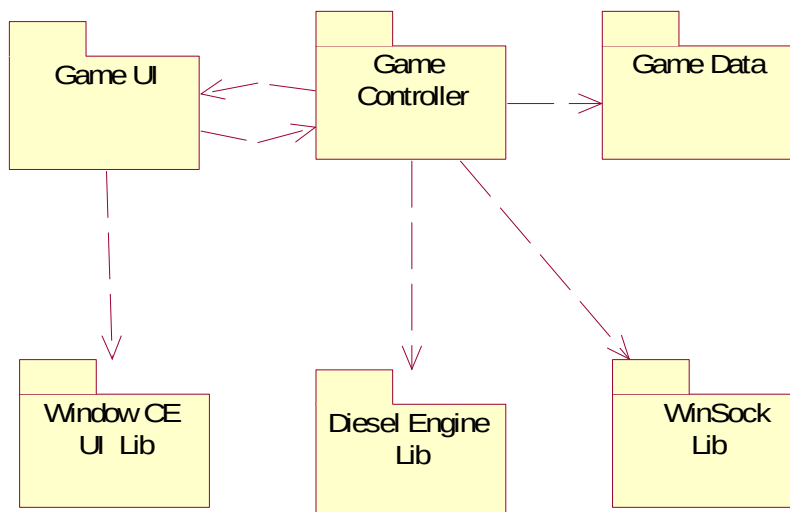
2. Class diagram ในมุมมองภายในเกม ระดับการออกแบบ (Design)



รูปที่ 3.14 แสดงคลาสไดอะแกรมของเกมในระดับการออกแบบระบบ

3.8 การออกแบบสถาปัตยกรรมภายในเกม

สถาปัตยกรรมของระบบในระดับของระบบเกม แสดงในภาพต่อไปนี้



รูปที่ 3.15 Package diagram แสดงสถาปัตยกรรมของระบบภายในเกม

ระบบของเกมมีการติดต่อระหว่างส่วนประกอบต่างๆ ดังนี้

1. Game Controller ทำหน้าที่ควบคุมดูแลจัดการการทำงานภายในระบบทั้งหมด เป็นเสมือนตัวกลางที่เชื่อมการทำงานระหว่างส่วนต่างๆ ภายในระบบ
2. Game Data ทำหน้าที่ในการจัดการส่วนของข้อมูลที่ใช้ภายในเกมทั้งหมด
3. Game UI ทำหน้าที่จัดการการติดต่อระหว่างผู้เล่นกับระบบตัวเกม
4. Window CE UI Lib ไบรารีของระบบปฏิบัติการบนอุปกรณ์ฟ็อกเก็ตพีซี ทำหน้าที่จัดการการกระทำของผู้เล่นที่กระทำกับอุปกรณ์ฟ็อกเก็ตพีซี แล้วเชื่อมข้อมูลให้กับ Game UI ในแอปพลิเคชัน
5. Winsock Lib ทำหน้าที่จัดการการเชื่อมต่อเครือข่าย และส่งข้อมูลผ่านเครือข่าย
6. Diesel Engine Lib ทำหน้าที่จัดการประมวลผลการแสดงผลภาพสามมิติและระบบเสียงที่ใช้ในแอปพลิเคชัน

การทำงานของระบบภายในเกมมีลักษณะการทำงาน ดังต่อไปนี้

1. เมื่อผู้เล่นเปิดเกมขึ้นมา ระบบเกมจะทำการโหลดรูปภาพและข้อมูลที่ต้องใช้ในเกม เช่น ข้อมูลโมเดลสามมิติ เข้าไปเก็บในหน่วยความจำ โดยระบบเกมใช้เกมเอนจินท์ 'Diesel Engine' ซึ่งมีหน้าที่ในการควบคุมดูแลจัดการข้อมูลและการแสดงผลสามมิติ เมื่อจัดการข้อมูลเบื้องต้นเสร็จแล้วจึงทำการแสดงผลไปยังหน้าจอ

2. เมื่อเริ่มเกมแล้ว ส่วน Game UI ทำหน้าที่รองรับการกระทำต่างๆ จากผู้เล่นผ่านทางหน้าจอติดต่อผู้ใช้ (User Interface)
3. เมื่อผู้เล่นกระทำการใดๆ ต่อหน้าต่างของเกม Input จากผู้เล่นจะผ่านอุปกรณ์แล้วส่วนของ Window CE UI Lib จะทำการประมวลผลส่งผลที่ได้ไปให้ส่วน Game UI แล้วส่งค่าที่ใช้ภายในแอปพลิเคชันไปให้ Game Controller เพื่อทำการประมวลผลแล้วจัดการส่งค่าไปยังส่วนการทำงานภายในระบบที่เกี่ยวข้องกับเมนู หรือคำสั่งที่ผู้เล่นได้กระทำในลำดับต่อไป
4. หากการทำงานส่วนใดเกี่ยวข้องกับข้อมูลที่ใช้ภายในแอปพลิเคชัน ส่วน Game Controller จะทำการส่งการทำงานไปให้ส่วน Game Data ทำการจัดการข้อมูลต่อ
5. ในกรณีที่ผู้เล่นเลือกเมนูการทำงานที่ต้องมีการติดต่อกับผู้เล่นคนอื่น หรือเครื่องเซิร์ฟเวอร์ ระบบในส่วน Game Controller จะทำการส่งการทำงานไปให้ส่วน Winsock Lib ทำการติดต่อผ่านเครือข่ายไร้สาย

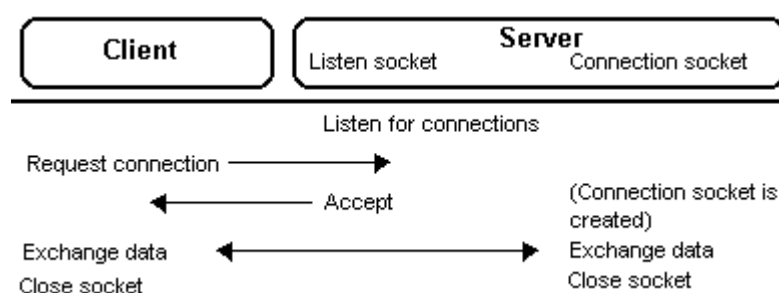
3.9 การออกแบบการเชื่อมต่อผ่านเครือข่ายโดยใช้ Winsock

3.9.1 สถาปัตยกรรมขั้นสูง(High Level Architecture)

รูปแบบในการออกแบบสถาปัตยกรรมขั้นสูงนั้นขึ้นอยู่กับว่าเกมที่เราก่อแบบมานั้นต้องการติดต่อกันในรูปแบบใดซึ่งรูปแบบทั่วไปที่นิยมใช้กันมีดังนี้

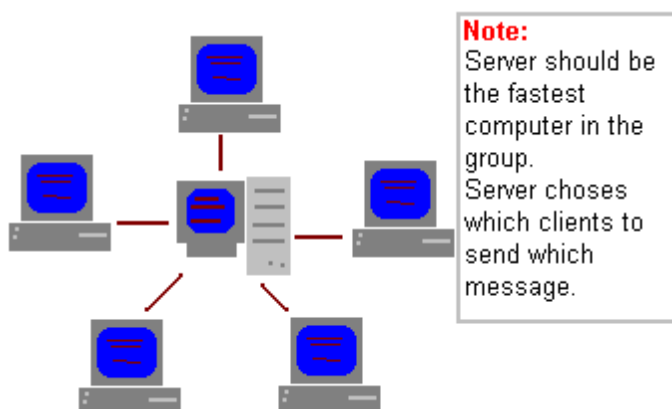
1. Client-server

ในรูปแบบนี้ผู้เล่นทุกคนหรือ client จะเชื่อมต่อไปยังเครื่องส่วนกลาง หรือ server ซึ่ง server จะทำหน้าที่ตัดสินใจข้อมูลที่สำคัญ ควบคุมสถานะ และทำการกระจายข้อมูลไปยัง client ซึ่งทำให้ระบบมีความคงทนเนื่องจากระบบถูกประมวลผลอยู่ในส่วนกลาง แต่ในบางระบบserver อาจไม่ทำการประมวลผลข้อมูลเลยก็ได้ขึ้นอยู่กับกรออกแบบของผู้พัฒนา จากการที่เราใช้ server เป็นส่วนกลางทำให้ server ต้องทำงานร่วมกับ client ทุกเครื่องที่ต่อเข้ามา ทำให้มีปัญหาที่ตามในด้านของระบบเครือข่ายเช่น Bottleneck แต่ข้อดีก็คือ server จะทำหน้าที่ประมวลผลแทน client ซึ่ง client มีหน้าที่เพียงแสดงผลเท่านั้น



รูปที่ 3.16 แสดงการจำลองรูปแบบการติดต่อระหว่าง Client กับ Server

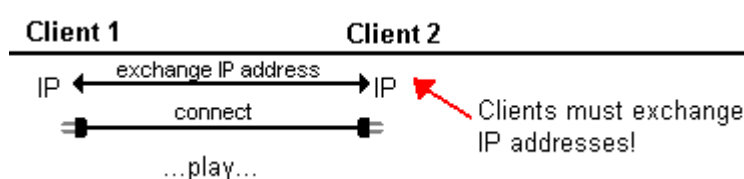
จากรูปจะเริ่มด้วย client สร้าง socket ขึ้นมา 1 socket เพื่อใช้ในการเชื่อมต่อ ส่วน Server ใช้ 2 socket socket ทำหน้าที่ในการรับฟังเพื่อรอรับการเชื่อมต่อ เมื่อได้รับแล้วจึงทำการสร้าง socket เพิ่ม คือ connection socket เพื่อป้องกันให้สามารถทำการเชื่อมต่อกับเครื่องอื่นๆ ได้



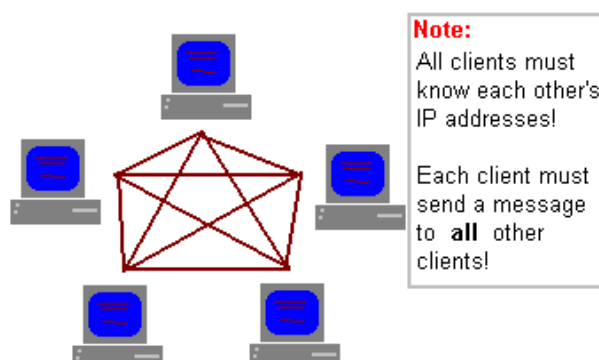
รูปที่ 3.17 แสดงการจำลองการเชื่อมต่อระหว่าง Client กับ Server

2. Client-Client

เป็นรูปแบบที่ทุกผู้เล่นต้องการประมวลผลและส่งข้อมูลในการตัดสินใจไปยังทุกเครื่องที่เล่นอยู่ ผลให้มีการเปลี่ยนแปลงทรัพยากรทั้งในการประมวลผลและ Bandwidth ที่ใช้ ซึ่งเมื่อต้องการการเชื่อมต่อจำเป็นต้องรู้ IP address ของอีกฝั่ง ซึ่งไม่สะดวกในการใช้งาน



รูปที่ 3.18 แสดงการจำลองรูปแบบการติดต่อระหว่าง Client กับ Client



รูปที่ 3.19 แสดงการจำลองการเชื่อมต่อระหว่าง Client กับ Client

3. Hybrid

เป็นวิธีที่ช่วยกันทำงานทั้งในส่วน Client และ server โดยให้ client สามารถทำการตัดสินใจในข้อมูลบางอย่างได้เช่นการเคลื่อนที่ แล้วจึงส่งข้อมูลอัปเดตไปยัง server ซึ่งเกมส่วนใหญ่จะใช้รูปแบบนี้ แต่ข้อเสียก็คือผู้เล่นสามารถโกงได้

3.9.2 สถาปัตยกรรมขั้นพื้นฐาน (Low Level Architecture)

1. TCP/IP

ในระดับนี้ส่วนใหญ่เกมในปัจจุบันมักมีตัวเลือก 2 ทางคือ TCP หรือ UDP
TCP หรือ Transmission control protocol เป็น Connection orient ซึ่งมีความน่าเชื่อถือในการส่งข้อมูล ส่วน UDP เป็นโปรโตคอลแบบ Connectionless กล่าวคือสามารถส่งได้รวดเร็ว แต่ข้อมูลขาดความน่าเชื่อถือเนื่องจากอาจเกิดการสูญหาย หรือชนกันของข้อมูลสูงกว่าโปรโตคอล TCP

2. โปรโตคอลในการติดต่อและรูปแบบข้อมูลที่ใช้

โดยรวมแล้วระบบของเราออกแบบให้ใช้รูปแบบ TCP ข้อมูลที่ส่งนั้นเป็นค่าข้อมูลประจำตัวมอนสเตอร์ของผู้เล่น ตัวเกมสตั๊กกับ เซิร์ฟเวอร์ มีการตกลงให้รับ-ส่ง ข้อมูลในรูปแบบเดียวกันเป็นดังนี้

ข้อมูลที่เซิร์ฟเวอร์รับ : STR#QI#AGI#DEF#VIT#DEX#FIT#BRE#INT#name

ข้อมูลที่เซิร์ฟเวอร์ส่งกลับ : name#STR#QI#AGI#DEF#VIT#DEX#FIT#BRE#INT#Lv#!

รูปแบบข้อมูลดังกล่าวเป็นรูปแบบที่ผู้พัฒนาได้กำหนดขึ้นเอง โดยตัวข้อมูลในไฟล์เป็นเซตของตัวอักษร (String) สำหรับข้อมูลภายในนั้นมีความหมาย คือ (ตัวอักษร) แทนค่าพารามิเตอร์นั้นๆ ของตัวมอนสเตอร์, เครื่องหมาย '#' หมายถึงหมดขอบเขตของค่าพารามิเตอร์นั้นๆ แล้ว และเครื่องหมาย '!' หมายถึง จบกระแสข้อมูลที่ผู้ส่งต้องการส่งแล้ว

ตัวอย่างโปรแกรมการทำงานอาจเป็นดังนี้

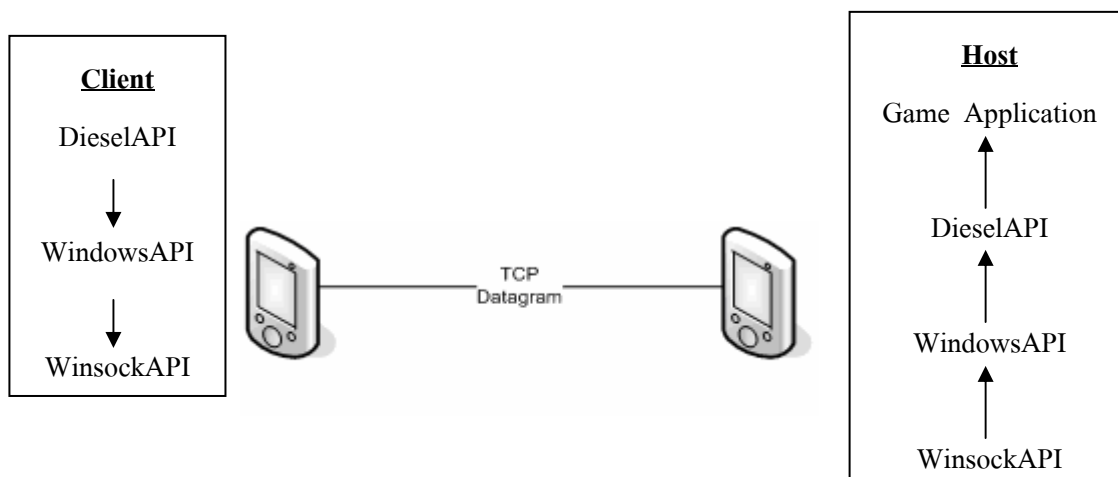
- การเชื่อมต่อระหว่างเครื่องผู้เล่น กับผู้เล่นคนอื่น
- เก็บข้อมูลตัวละครมอนสเตอร์ของผู้เล่นเอง
- รับข้อมูลมอนสเตอร์ของผู้เล่นคนอื่น
- ทำการสร้างโมเดลของมอนสเตอร์ของผู้เล่นคนอื่นตามข้อมูลที่ได้รับ
- ทำการ 'ต่อสู้อย่างมอนสเตอร์' (ไม่ใช้การเชื่อมต่อกับเครือข่าย)

- งานทำงานในระบบการต่อสู้จนกว่าจะรู้ผลแพ้ - ชนะ
- อัปเดตสถานะของผู้เล่นตามผลการต่อสู้
- ส่งข้อมูลมอนสเตอร์คืนแก่ผู้เล่นที่เป็นเจ้าของ

3.9.3 สถาปัตยกรรมการเชื่อมต่อโดยรวม

1. การเชื่อมต่อระหว่าง client – client

การเชื่อมต่อระหว่าง 2 เครื่องนั้นเครื่องหนึ่งทำหน้าที่เป็น Host ทำหน้าที่ในการเปิดการติดต่อ และอีกเครื่องหนึ่งเป็น client เข้ามาทำการเชื่อมต่อ เมื่อมีการเริ่มต้นการติดต่อโดย host จะทำการเปิด socket โดยเริ่มจากรับคำสั่งจากหน้าจอผ่าน DieselAPI ส่งผ่าน WinAPI ไปยัง WinsockAPI เพื่อเปิดการติดต่อ socket สำหรับเป็นเส้นทางในการส่ง TCP Datagram ไปยังปลายทาง โดยที่ในส่วนของ client มีการทำงานเช่นเดียวกัน



รูปที่ 3.20 แสดงการจำลองการเชื่อมต่อระหว่าง Client กับ Client

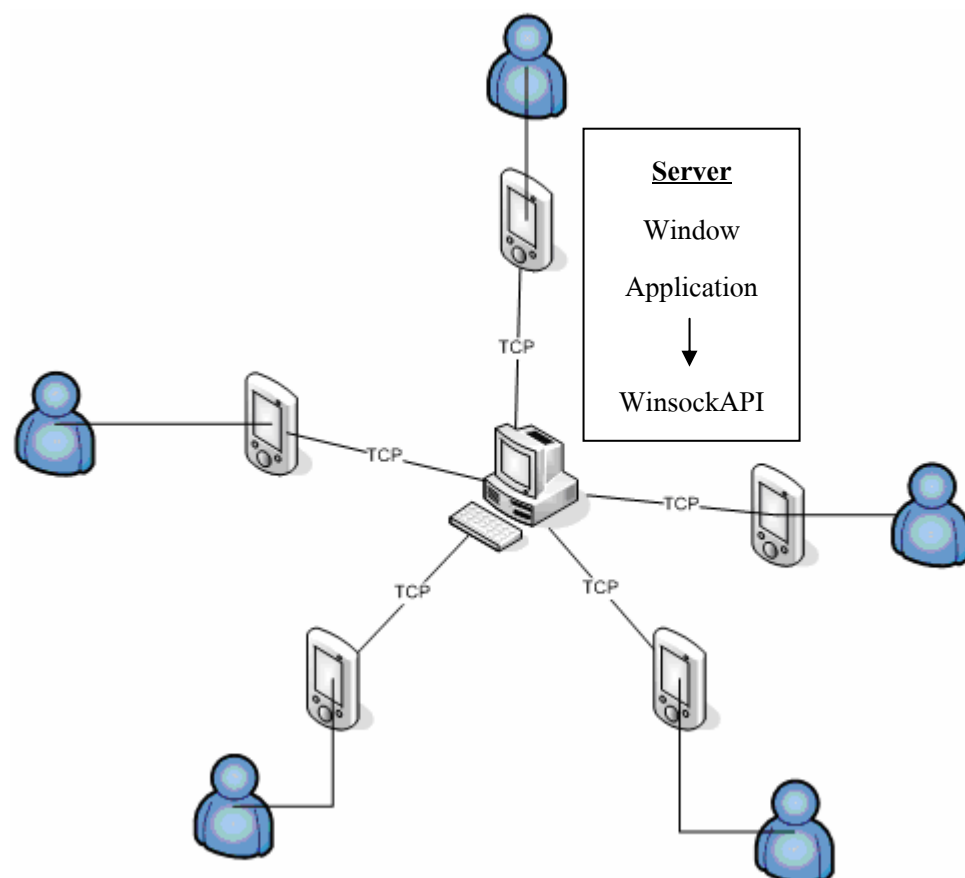
2. การเชื่อมต่อระหว่าง client – server

การเชื่อมต่อระหว่าง Client – server เครื่อง server ทำหน้าที่ในการเปิดการติดต่อให้เครื่อง client เข้ามาทำการเชื่อมต่อ เมื่อมีการเริ่มต้นการติดต่อโดย server จะทำการเปิด socket และรอรับการติดต่อจาก client โดยการติดต่อนั้นต้องสถาปนาการเชื่อมต่อให้เสร็จสิ้นเสียก่อน เพราะเป็นการติดต่อแบบ Connection Oriented

เครื่อง Client รับคำสั่งของผู้เล่นจากหน้าจอเกมผ่าน DieselAPI ส่งผ่าน WinAPI ไปยัง WinsockAPI เพื่อเปิดการติดต่อ socket สำหรับเป็นเส้นทางในการส่ง TCP Datagram ซึ่งบรรจุข้อมูลตัวละครของผู้เล่นส่งไปยังเครื่อง server ปลายทาง

เครื่อง Server ทำการรับข้อมูลจากเครื่อง client แต่ละเครื่องผ่านทาง socket ซึ่งจัดการโดย Winsock API จากนั้นส่งข้อมูลที่รับมาไปยัง Window Application เพื่อทำการประมวลผล ซึ่งการ

อ้างอิงและบันทึกข้อมูลทั้งหมดจะกระทำกับฐานข้อมูลของระบบที่ Database Server และในทุกครั้งที่มีการรับข้อมูลเข้ามาใหม่จะทำการอัปเดตข้อมูลนั้นๆ ไปยังเครื่อง client ที่ทำการเชื่อมต่ออยู่ทั้งหมด



รูปที่ 3.21 แสดงการจำลองการเชื่อมต่อระหว่าง Client กับ Server